UNIVERSITATEA BABEŞ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

LUCRARE DE LICENŢĂ

Optimizarea prin preprocesare adaptivă a sistemelor de recunoaștere facială bazate pe rețele neuronale cu spiking

Conducător științific

Drd. Marc Daria
Prof. Dr. Pop Horia Florin

Absolvent
Bubuianu Adrian-Florin

2026

ABSTRACT

Abstract: un rezumat în limba engleză cu prezentarea, pe scurt, a conținutului pe capitole, punând accent pe contribuțiile proprii și originalitate

Cuprins

Introducere **1**

Încadrarea în literatura de specialitate **2**
0.1 Recunoașterea emoțiilor faciale folosind rețele SNN bio-inspirate . . 2
0.2 Abordări bazate pe evenimente: Arhitectura Spiking-FER 2
0.3 Fuziunea multimodală și arhitecturi evolutive: Modelul FusionSense 3
0.4 Recunoașterea identității prin dinamica mișcărilor faciale 4
0.5 Recunoașterea emoțiilor prin senzori de viziune dinamică simulați . 4

1 Fundamente teoretice despre inteligența artificială și rețele neuromorfice **6**
1.1 Învățarea automată și rețele neuronale artificiale (ANN) 6
1.2 Trecerea de la ANN la Spiking Neural Networks (SNN) 7
1.3 Modelarea matematică a neuronului cu impulsuri 8
1.3.1 Dinamica membranei neuronale 8
1.3.2 Modele de neuroni: de la biologie la eficiență hardware 9
1.3.3 Formalizarea matematică a modelului Integrate-and-Fire . . . 10
1.4 Codificarea informației neurale (Neural Coding) 11
1.4.1 Rate Coding 11
1.4.2 Temporal Coding 11
1.5 Mecanisme de învățare și plasticitate sinaptică 12
1.5.1 Conversia ANN în SNN 13
1.5.2 Spiking Back-propagation 13
1.5.3 Învățarea de tip Hebbian 14
1.5.4 Spike Timing-Dependent Plasticity (STDP) 14
1.5.5 Inhibiția 15
1.5.6 Homeostazia 16
1.6 Arhitecturi Spiking Convolutional Neural Networks (CSNN) 17
1.6.1 Convoluția în domeniul impulsurilor (2D-CSNN) 17
1.6.2 3D-CSNN pentru procesare video 18
1.7 Algoritmi de preprocesare și detecție facială 18
1.7.1 Detecția feței prin algoritmul Viola-Jones 19

1.7.2 Extragerea și urmărirea punctelor de interes 19
1.7.3 Transformări geometrice și alinierea feței 20
1.8 Metode de eșantionare a datelor de intrare (Sampling) 20
1.8.1 Eșantionarea Spațială (2D) 21
1.8.2 Eșantionarea Spatio-Temporală (3D) 22
1.9 Metode de Decodificare a Semnalelor Neuronale 24
1.9.1 Decodificarea prin Declin Liniar (Linear Decay) 24
1.9.2 Decodificarea prin Impuls Dreptunghiular (Rectangular Pulse) 25
1.9.3 Decodificarea Gaussiană (Gaussian-based Importance) 25
1.10 Percepția vizuală bazată pe evenimente 25
1.10.1 Senzori de viziune dinamică (DVS) 25
1.10.2 Recunoașterea activităților umane (HAR) folosind SNN . . . 25

2 Recunoașterea facială și procesarea semnalului vizual 26
2.1 Biometria facială: provocări și oportunități 26
2.2 Reprezentarea datelor sub formă de impulsuri (Spike Encoding) . . . 26
2.2.1 Rate Coding vs. Temporal Coding 26
2.3 Problematica zgomotului de fundal în sistemele neuromorfice 26

3 Preprocesarea imaginii și segmentarea Regiunii de Interes (ROI) 28
3.1 Importanța segmentării în eficientizarea CSNN 28
3.2 Algoritmi pentru detecția feței 28
3.3 Extragerea ROI și normalizarea pentru conversia în impulsuri 28
3.4 Reducerea activității neuronale redundante prin eliminarea fundalului 28

4 Proiectarea și implementarea sistemului de recunoaștere facială 30
4.1 Arhitectura sistemului propus (Pipeline-ul de procesare) 30
4.1.1 Simulatorul CSNN 30
4.2 Tehnologii și biblioteci utilizate 30
4.2.1 Biblioteci de procesare vizuală (OpenCV) 30
4.3 Configurarea straturilor convoluționale ale CSNN 30
4.4 Mecanismul de antrenare 30

5 Experimente și analiza rezultatelor 31
5.0.1 Analiza Arhitecturilor de Clasificare în CSNN 31
5.1 Setul de date utilizat 32
5.2 Metodologia de testare a modelului 32
5.2.1 Cazul A: Recunoaștere pe imagini brute (cu fundal) 32
5.2.2 Cazul B: Recunoaștere pe imagini cu segmentare ROI 32
5.3 Metrici de performanță 32

5.4 Analiza impactului zgomotului vizual asupra preciziei modelului . . 32

6 Numele aplicației 33

7 Colectarea cerințelor 34
7.1 Actorii sistemului . . . 34
7.2 Diagrama cazurilor de utilizare . . . 34
7.3 Descrierea detaliată a unui caz de utilizare . . . 35
7.4 Cerințe nefuncționale (NFR) . . . 35
7.4.1 Securitate și control acces . . . 35
7.4.2 Confidențialitate și protecția datelor . . . 37
7.4.3 Performanță și timp real . . . 37
7.4.4 Fiabilitate și disponibilitate . . . 37
7.4.5 Uzabilitate și Mentenabilitate . . . 38
7.5 Modelul structural . . . 38
7.5.1 Backend . . . 38
7.5.2 Frontend . . . 40
7.5.3 Diagrama entitate–relație (ER) . . . 42
7.6 Modelul dinamic al sistemului . . . 43
7.6.1 Diagrama de secvență: Monitorizare live . . . 43
7.6.2 Diagrama de comunicare: Vizualizare timeline . . . 44
7.6.3 Diagrama de deployment . . . 45
7.6.4 Diagrama de stare: *PatientAlert* . . . 47
7.6.5 Diagrama de activități: Check-in pacient . . . 49
7.7 Sistemul de alerte și notificări . . . 49
7.7.1 Evenimente clinice în sesiune (Live Session Events) . . . 51
7.7.2 Indicatori rezumați în UI . . . 51
7.7.3 Notificare la schimbarea nivelului de risc . . . 53
7.7.4 Notificări operaționale (erori și stări de funcționare) . . . 53
7.8 Testarea aplicației . . . 54
7.8.1 Testare white-box . . . 54
7.9 Testarea de integrare . . . 55
7.9.1 Testare de integrare (Top-down, Depth-first) . . . 55
7.10 Manual de utilizare al aplicației . . . 59
7.10.1 Scopul aplicației și roluri . . . 59
7.10.2 Accesarea aplicației . . . 60
7.10.3 Autentificare și creare cont . . . 60
7.10.4 Modul ADMIN: Administrative Dashboard . . . 60
7.10.5 Modul DOCTOR: Doctor Panel și Monitoring Dashboard . . . 61
7.10.6 Confidențialitate și bune practici . . . 64

7.11 Proiectare 64
7.11.1 Analiza cerințelor 64
7.11.2 Diagrama cazurilor de utilizare 64
7.11.3 Diagrame de interacțiune 64
7.12 Implementare 64
7.12.1 Server 64
7.12.2 Client 64
7.13 Testare 64

8 Concluzii **65**

Bibliografie **66**

Introducere

Introducere: obiectivele lucrării și descrierea succintă a capitolelor, prezentarea temei, prezentarea contribuției proprii, respectiv a rezultatelor originale și menționarea (dacă este cazul) a sesiunii de comunicări unde a fost prezentată sau a revistei unde a fost publicată.

Încadrarea în literatura de specialitate

0.1 Recunoașterea emoțiilor faciale folosind rețele SNN bio-inspirate

Lucrarea elaborată de Mansouri-Benssassi și Ye [MBY21] investighează aplicabilitatea rețelelor neuronale cu impulsuri (SNN) bio-inspirate în sarcini de recunoaștere a emoțiilor faciale (FER), punând un accent deosebit pe robustețea la zgomot și capacitatea de generalizare. Arhitectura propusă adaptează modelul Diehl și Cook [DC15a], utilizând învățarea nesupervizată prin plasticitate dependentă de timpul impulsurilor (STDP). Procesarea imaginilor implică utilizarea filtrului Laplacian of Gaussian (LoG) pentru extracția eficientă a marginilor, urmată de o codificare de tip *rate coding* prin trenuri de impulsuri Poisson, unde intensitatea pixelilor dictează rata de declanșare a neuronilor de tip Leaky Integrate-and-Fire (LIF). Modelul este structurat pe trei straturi: unul de intrare, un strat excitator (cu funcții de convoluție) și un strat inhibitor pentru selecția neuronului activ. Evaluările realizate pe seturile de date JAFFE și CK+ relevă faptul că SNN-ul obține performanțe comparabile cu metodele HOG+SVM și superioare arhitecturilor CNN pe seturi de date mici. Un aspect critic evidențiat este superioritatea SNN în experimentele de generalizare *cross-dataset* (atingând până la 92%) și reziliența remarcabilă la zgomot de tip *salt and pepper*; la un nivel de degradare de 0,5, SNN menține o acuratețe de 56,2%, depășind net modelele CNN (22,6%) și SVM (14,1%). Totuși, autorii identifică limitări privind utilizarea codificării pe bază de rată în detrimentul celei temporale și necesitatea rulării pe hardware neuromorfic dedicat pentru a maximiza eficiența energetică și computațională față de implementările actuale pe CPU/GPU.

0.2 Abordări bazate pe evenimente: Arhitectura Spiking-FER

O contribuție majoră în domeniu este adusă de Barchid et al. [BAA+23], care introduc conceptul de „Event-based FER" și propun primul benchmark de acest tip prin

conversia seturilor de date video clasice (CK+, ADFES, CASIA și MMI) în fluxuri de evenimente folosind simulatorul V2E [HLD21]. Autorii propun *Spiking-FER*, o arhitectură SNN convoluțională profundă bazată pe SEW-ResNet-18, care utilizează neuroni de tip *Integrate-and-Fire* (IF). Pentru a permite antrenarea de tip *end-to-end* și a depăși problema non-diferențiabilității funcției de prag, se utilizează *Surrogate Gradient Learning* cu o aproximare bazată pe funcția arctan. Spre deosebire de metodele modulare, abordarea *end-to-end* permite optimizarea simultană a extractorului de trăsături și a clasificatorului. Rezultatele experimentale indică faptul că *Spiking-FER* atinge performanțe comparabile cu modelele ANN, însă cu o eficiență energetică net superioară, fiind de până la 65,39 ori mai eficient din punct de vedere computațional. Studiul analizează, de asemenea, impactul tehnicilor de augmentare a datelor specifice evenimentelor (*Event Data Augmentation*), concluzionând că metodele *EventDrop* și *Mirror* sunt esențiale pentru reglarea antrenării pe seturi de date limitate. Totuși, o limitare importantă a studiului rămâne dependența de date simulate în locul senzorilor DVS (*Dynamic Vision Sensor*) reali și utilizarea bazelor de date de laborator, care pot să nu reflecte complexitatea emoțiilor spontane din mediul real.

0.3 Fuziunea multimodală și arhitecturi evolutive: Modelul FusionSense

Lucrarea prezentată de Tan et al. [TCKPS20] introduce *FusionSense*, primul sistem de recunoaștere a emoțiilor bazat pe rețele neuronale cu impulsuri care utilizează fuziunea datelor multimodale (expresii faciale corelate cu semnale fiziologice precum ECG, conductanța pielii și respirația). Autorii utilizează cadrul *NeuCube* [Kas14], o arhitectură SNN evolutivă (eSNN) care încorporează o structură spațială 3D de tip *reservoir* (3D-SNNr) cu conectivitate *small-world*. Pipeline-ul vizual utilizează algoritmul Viola-Jones pentru detecție și metoda Kanade-Lucas-Tomasi (KLT) pentru urmărirea punctelor de interes, extrăgând distanțe relative între 68 de repere faciale (*landmarks*) raportate la o stare neutră de referință. Un aspect metodologic distinct este utilizarea codificării de tip *population coding* bazată pe câmpuri receptive Gaussiene, urmată de o învățare nesupervizată prin STDP și o etapă supervizată ce folosește reguli de tip *Rank Order* (deSNN). Experimentele pe setul de date MAHNOB-HCI demonstrează că fuziunea la nivel de trăsături (*feature-level fusion*) este superioară celei la nivel de decizie, atingând o acuratețe de 73,15% pentru valența binară. Sistemul se remarcă prin capacitatea de antrenare incrementală (*one-pass*), fiind optim pentru aplicații *real-time*. Totuși, studiul prezintă limitări legate de excluderea semnalelor EEG și de dependența de o fereastră temporală fixă de 30 de

secunde, care poate omite dinamica emoțională pe termen lung.

0.4 Recunoașterea identității prin dinamica mișcărilor faciale

Lucrarea propusă de Liu et al. [LBY20] introduce o paradigmă inovatoare de recunoaștere facială bazată pe dinamica mișcărilor musculare, metodă eficientă chiar și în scenarii cu fețe deghizate sau parțial ocultate. Spre deosebire de sistemele convenționale care se bazează pe trăsături spațiale statice, acest model utilizează cadre diferențiale pentru a captura micro-mișcările unice induse de vorbire. Arhitectura este de tip SNN profund, structurată pe cinci straturi care simulează calea ventrală a sistemului vizual biologic, incluzând module de extracție a trăsăturilor de nivel înalt prin filtre Gabor și codificare temporală bazată pe *Rank Order Coding* (ROC). Un element central al modelului este stratul de învățare care utilizează mecanismul *Soft Winner-Take-All* (WTA) cu inhibiție laterală și reguli STDP *event-driven*, asigurând selectivitatea neuronilor pentru tipare motorii specifice. Evaluările pe setul de date MakeFaceDB au demonstrat rate de succes între 95% și 100%, depășind modelele CNN echivalente, în special în scenariile cu deghizări complexe (măști, barbă falsă). Autorii subliniază eficiența procesului de convergență, care necesită doar două clipuri video pentru antrenare. Totuși, sunt identificate limitări practice importante: dependența de mișcarea activă a subiectului (necesitatea de a vorbi), sensibilitatea ridicată la mișcările de fundal sau ale camerei și necesitatea unei reglări manuale riguroase a hiperparametrilor pentru a menține stabilitatea rețelei.

0.5 Recunoașterea emoțiilor prin senzori de viziune dinamică simulați

Lucrarea propusă de Wang et al. [WDZ+22] abordează deficitul de seturi de date pentru senzori de viziune dinamică (DVS) în recunoașterea emoțiilor prin propunerea unei metode de generare a datelor simulate din segmente video convenționale (setul CMU-MOSI [ZZPM16]). Autorii introduc un cadru SNN bazat pe modelul de neuron *Parametric Leaky Integrate-and-Fire* (PLIF), care permite optimizarea automată a parametrilor de scurgere (*leak*) în timpul antrenării. O contribuție metodologică importantă este utilizarea unei prime rame de tip *float-value* pentru a stabiliza potențialul de membrană, urmată de trenuri de impulsuri binare, strategie care accelerează scăderea funcției de pierdere și îmbunătățește acuratețea față de modelele care folosesc exclusiv *spikes*. Arhitectura este compusă dintr-un modul de extracție

a trăsăturilor cu straturi convoluționale și *average pooling*, un grup de neuroni de votare și un modul de mapare emoțională care traduce ratele de declanșare în etichete de polaritate. Antrenarea se realizează prin gradient surogat și *backpropagation through time* (BPTT). Rezultatele experimentale demonstrează că modelul SNN reduce consumul de energie cu 75% față de echivalentul ANN, atingând performanțe optime la un prag de tensiune de 0,4. Totuși, datele simulate nu pot reproduce fidel rezoluția temporală fină a unui senzor DVS real, iar constrângerea ferestrei temporale la 68 de cadre poate duce la pierderea informațiilor în cazul secvențelor video de lungimi inegale.

Capitolul 1

Fundamente teoretice despre inteligența artificială și rețele neuromorfice

1.1 Învățarea automată și rețele neuronale artificiale (ANN)

Învățarea automată (*Machine Learning*) constituie pilonul central al inteligenței artificiale contemporane, oferind sistemelor computaționale capacitatea de a identifica tipare complexe în seturi de date fără a necesita o programare algoritmică explicită pentru fiecare regulă de decizie. În centrul acestei paradigme se află rețelele neuronale artificiale (ANN), arhitecturi proiectate ca o abstractizare a rețelelor neuronale biologice, însă simplificate structural pentru a fi compatibile cu hardware-ul convențional.

Funcționarea unei rețele neuronale standard se bazează pe transmiterea informației printr-o ierarhie de straturi compuse din unități de procesare elementare. Fiecare neuron artificial realizează o transformare a datelor de intrare în două etape succesive. În prima fază, se calculează o combinație liniară a semnalelor de intrare, unde fiecare contribuție este ajustată prin intermediul unor coeficienți numiți ponderi (sau greutăți). Rolul acestora este de a amplifica sau atenua importanța fiecărui semnal în procesul decizional. Matematic, această sumă ponderată, la care se adaugă un termen de polarizare (*bias*), este exprimată prin relația (1.1):

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{1.1}$$

unde x_i reprezintă intrările sistemului, w_i sunt ponderile asociate, iar b este termenul de *bias*. În a doua etapă, pentru a introduce neliniaritatea necesară modelării unor fenomene complexe, rezultatul z este procesat de o funcție de activare σ. Va-

loarea finală de ieșire a neuronului, y, este definită de ecuația (1.2), procesul fiind ilustrat schematic în Figura 1.1:

$$y = \sigma(z) = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{1.2}$$

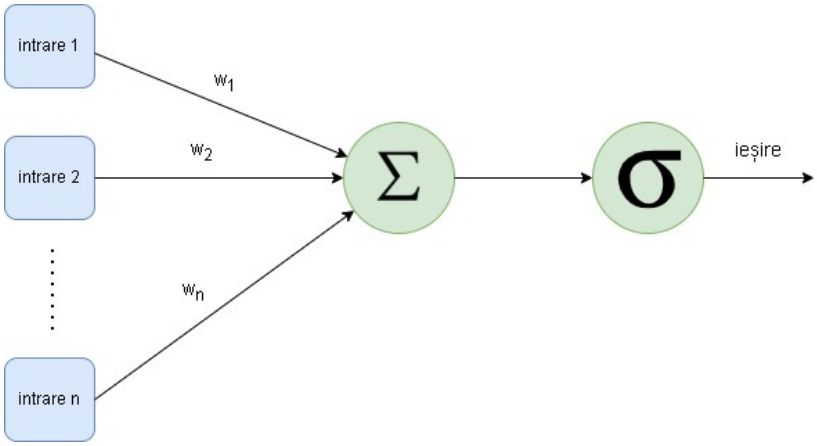


Figura 1.1: Reprezentarea schematică a unui neuron artificial: procesarea intrărilor prin ponderare și aplicarea funcției de activare.

Deși aceste rețele, integrate în ceea ce comunitatea științifică numește „a doua generație" de modele neuronale, au demonstrat performanțe remarcabile, ele prezintă limitări inerente. Utilizarea valorilor numerice continue, conform modelului descris în ecuația (1.2), forțează sistemul să realizeze calcule intensive în mod constant. Acest aspect conduce la un cost computațional ridicat și la un consum energetic substanțial, care rămâne invariant indiferent de relevanța sau densitatea informației procesate.

1.2 Trecerea de la ANN la Spiking Neural Networks (SNN)

Limitările energetice menționate anterior au impus tranziția către o nouă paradigmă: rețelele neuronale cu impulsuri (*Spiking Neural Networks* - SNN). Considerate a treia generație a rețelelor neuronale, SNN-urile se apropie mult mai mult de mecanismele fiziologice ale creierului, înlocuind valorile continue cu impulsuri discrete de joasă energie, numite *spikes* [TGK+19].

Spre deosebire de modelele clasice, SNN-urile adoptă o abordare bazată pe evenimente (*event-driven*), caracterizată printr-o activare rară (*sparse activation*), așa cum se poate observa în Figura 1.2. În acest context, neuronii procesează evenimente distribuite temporal, activându-se doar atunci când stimulii recepționați ating un

anumit prag critic. Această proprietate elimină necesitatea calculelor redundante în perioadele de inactivitate, optimizând consumul de resurse [FTB+19]. Mai mult, natura asincronă a acestor rețele facilitează paralelizarea proceselor, oferind avantaje clare de viteză pe hardware neuromorfic dedicat [BBJ+19].

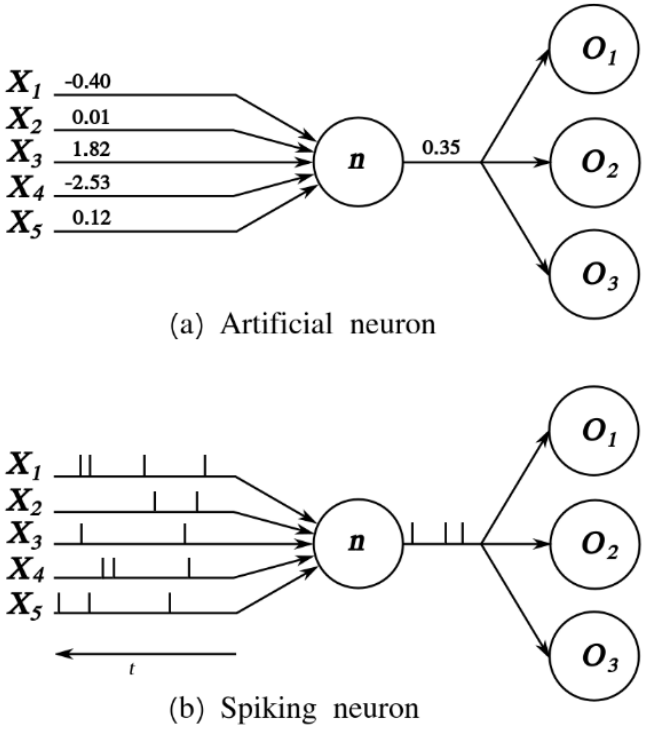


Figura 1.2: A comparison between an artificial neuron and a spiking neuron. The structure is common between ANNs and SNNs. However, (a) an artificial neuron receives numerical values to compute an output numerical value, while (b) a spiking neuron receives sparse events called spikes and generates a sequence of output spikes. Sursă: [EA24].

1.3 Modelarea matematică a neuronului cu impulsuri

Spre deosebire de modelele generațiilor anterioare, rețelele neuronale cu impulsuri (SNN) utilizează ca unitate fundamentală de comunicare fluxuri de impulsuri discrete, cunoscute sub denumirea de *spike trains*. Fiecare impuls reprezintă un potențial de acțiune, o descărcare electrică de scurtă durată care transportă informația prin rețea, imitând fidel procesele electro-chimice din creierul uman.

1.3.1 Dinamica membranei neuronale

Mecanismul de funcționare al unui neuron de tip *spiking* este definit prin interacțiunea dintre potențialul de membrană și un prag de activare (*firing threshold*). Procesul poate fi sintetizat în patru etape critice:

- **Integrarea și acumularea:** Neuronul recepționează semnale de intrare prin sinapse și le cumulează în timp, ceea ce determină o creștere incrementală a potențialului de membrană.
- **Declanșarea (*Firing*):** În momentul în care potențialul acumulat atinge sau depășește valoarea pragului critic stabilit, neuronul generează un impuls de ieșire (*spike*).
- **Resetarea:** Imediat după generarea impulsului, potențialul de membrană revine la o valoare de bază, numită stare de repaus (*resting state*).
- **Perioada refractară:** Urmează un scurt interval de timp în care neuronul devine insensibil la noile impulsuri primite, indiferent de intensitatea acestora, permițând refacerea echilibrului ionic.

Evoluția temporală a acestor etape și modul în care potențialul de membrană reacționează la stimulii externi sunt ilustrate în Figura 1.3.

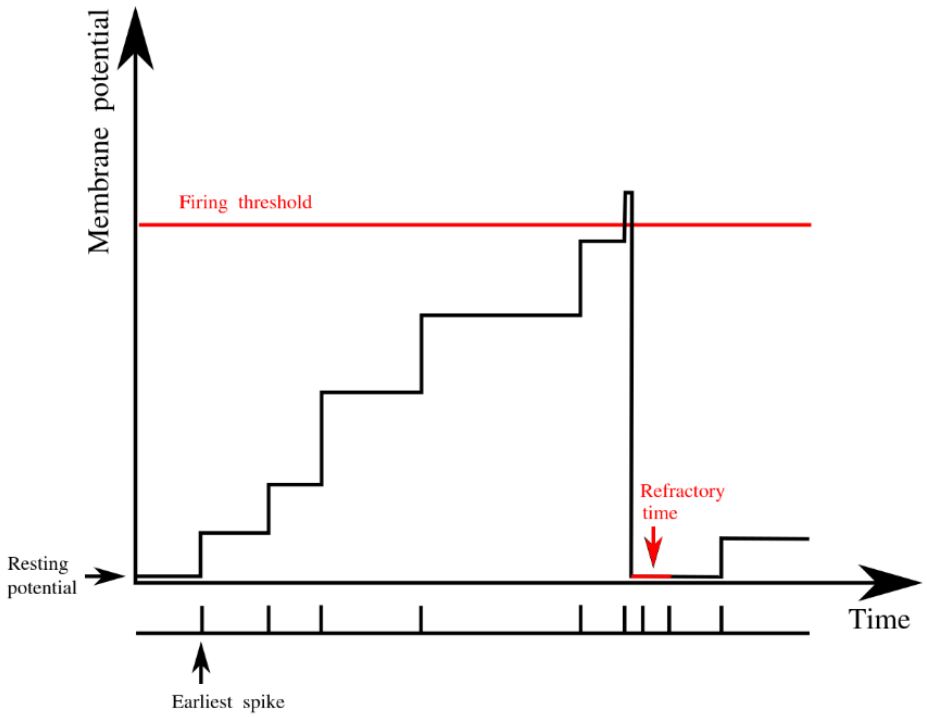


Figura 1.3: Evoluția potențialului de membrană al unui neuron de tip *spiking* ca răspuns la un tren de impulsuri de intrare. Se observă fazele de acumulare, atingerea pragului critic, generarea impulsului și perioada de resetare. Sursă: [EA24].

1.3.2 Modele de neuroni: de la biologie la eficiență hardware

În literatura de specialitate, există o varietate de modele matematice care descriu acest comportament, prezentând un compromis direct între plauzibilitatea biologică

și complexitatea computațională [Izh04].

1. **Modelul Hodgkin-Huxley [HH52]:** Este unul dintre cele mai detaliate modele biofizice, utilizând un sistem de ecuații diferențiale neliniare pentru a descrie dinamica canalelor ionice (sodiu și potasiu). Deși oferă o acuratețe biologică ridicată, costul computațional necesar pentru simularea acestuia îl face extrem de dificil de implementat în sisteme hardware neuromorfice la scară largă.

2. **Modelul Integrate-and-Fire (IF) [Bur06]:** Reprezintă varianta cea mai simplificată și eficientă din punct de vedere algoritmic. În ciuda arhitecturii sale elementare, modelul IF s-a dovedit a fi un instrument robust în aplicațiile de învățare vizuală și recunoaștere a formelor, fiind preferat în designul sistemelor de tip *edge AI*.

1.3.3 Formalizarea matematică a modelului Integrate-and-Fire

Modelul IF descrie evoluția potențialului de membrană $V_m(t)$ ca o acumulare liniară a ponderilor sinaptice asociate impulsurilor recepționate. Dinamica sistemului, guvernată de capacitatea membranei c_m, este exprimată prin relația (1.3):

$$c_m V_m(t) = V_r + \sum_{i \in E} w_i f_s(t - t_i) \tag{1.3}$$

unde:

- V_r reprezintă potențialul de repaus al membranei;
- E constituie setul de impulsuri de intrare;
- w_i este ponderea sinaptică (greutatea) asociată celui de-al i-lea impuls;
- t_i reprezintă coordonata temporală a recepționării impulsului;
- f_s este funcția nucleu (*spike kernel*), definită conform ecuației (1.4):

$$f_s(x) = \begin{cases} 1, & \text{dacă } x \geq 0 \\ 0, & \text{în caz contrar} \end{cases} \tag{1.4}$$

Regula de actualizare a stării neuronului impune ca atunci când potențialul de membrană depășește pragul V_{th}, să aibă loc descărcarea, urmată de resetarea potențialului:

$$V_m(t) \leftarrow V_r \quad \text{pentru} \quad V_m(t) \geq V_{th}(t) \tag{1.5}$$

Această abordare matematică permite procesarea eficientă a datelor asincrone, fiind ideală pentru sarcini care implică o dinamică temporală complexă, cum este identificarea emoțiilor în secvențe video.

1.4 Codificarea informației neurale (Neural Coding)

Întrucât rețelele neuronale cu impulsuri procesează informația sub formă de evenimente discrete, transformarea datelor de intrare (semnale analogice sau valori digitale continue) în trenuri de impulsuri (*spike trains*) reprezintă o etapă fundamentală. În contextul prezentei lucrări, datele de intrare sunt prezentate neuronilor secvențial, eșantion cu eșantion. Fiecare stimul este menținut pentru un interval temporal definit, cunoscut sub numele de perioadă de expunere sau $t_{exposition}$ [Fal19].

Deși există numeroase scheme de codificare, acestea pot fi clasificate în două categorii majore, în funcție de modul în care informația este reprezentată în raport cu timpul.

1.4.1 Rate Coding

Codificarea prin rată corelează intensitatea stimulului de intrare $x \in [0, 1]$ cu frecvența de apariție a impulsurilor. Din punct de vedere computațional, acest proces este modelat printr-un proces stochastic Poisson [GK02]. Probabilitatea ca un neuron i să emită un impuls ($S_i = 1$) într-un interval de timp discret foarte mic Δt este proporțională cu valoarea semnalului de intrare:

$$P(S_i[t] = 1) = x \cdot \Delta t \tag{1.6}$$

unde x reprezintă intensitatea stimulului (rata de descărcare dorită), Δt este pasul de simulare temporală, $S_i[t] \in \{0, 1\}$ este variabila Bernoulli care indică prezența unui impuls la momentul t.

Deși este robustă, codificarea prin rată prezintă limitări semnificative în ceea ce privește viteza de procesare. Pentru a obține o reprezentare precisă a valorii codificate, neuronii post-sinaptici trebuie să integreze un număr ridicat de impulsuri pe o durată extinsă [GFES21]. Acest fenomen introduce o latență inerentă în rețea, deoarece decizia este condiționată de acumularea unei statistici suficiente a impulsurilor.

1.4.2 Temporal Coding

Spre deosebire de codificarea prin rată, metodele temporale utilizează momentul precis al apariției impulsurilor (*timestamps*) pentru a reprezenta informația, oferind o eficiență energetică și temporală superioară.

Rank Order Coding

Codificarea prin ordinea rangului [TG98] se bazează pe premisa că ordinea în care neuronii dintr-o populație declanșează impulsuri poartă informația esențială [DPT01].

Neuronii care primesc stimuli de intensitate ridicată tind să se activeze înaintea celor stimulați slab. O formă simplificată de reprezentare a acestui impact, independentă de modelul de neuron utilizat, este redată de ecuația (1.7):

$$f_{in}(x_i) = r(x_i) \times \frac{1}{n} \tag{1.7}$$

unde x_i este valoarea de intrare, n reprezintă numărul total de conexiuni, iar $r(x_i)$ reprezintă rangul (poziția în ordine descrescătoare) al valorii x_i în setul de intrări.

Latency Coding

Codificarea prin latență [GM08] reprezintă intensitatea stimulului prin intervalul de timp scurs până la primul impuls. Un stimul puternic va genera un răspuns rapid (latență mică), în timp ce un stimul slab va întârzia activarea neuronului. Relația de inversă proporționalitate între intensitatea x și momentul declanșării este formalizată prin ecuația (1.8):

$$f_{in}(x) = (1.0 - x) \times t_{exposition} \tag{1.8}$$

Această abordare este deosebit de eficientă pentru procesarea video (HAR), deoarece permite rețelei să reacționeze extrem de rapid la schimbările semnificative din câmpul vizual, procesând prioritar cele mai relevante informații.

1.5 Mecanisme de învățare și plasticitate sinaptică

Capacitatea de procesare a informației în cadrul rețelelor SNN rezidă în conexiunile sinaptice dintre neuroni. **Plasticitatea sinaptică** reprezintă proprietatea fundamentală a acestor conexiuni de a-și modifica intensitatea (ponderea sinaptică) în funcție de tiparele activității neuronale și de regulile de învățare aplicate. Din punct de vedere funcțional, ponderile sinaptice modulează probabilitatea de declanșare a neuronului post-sinaptic: o pondere ridicată amplifică contribuția unui impuls recepționat, în timp ce o pondere scăzută atenuează impactul acestuia asupra potențialului de membrană.

Antrenarea acestor rețele presupune ajustarea sistematică a ponderilor și a pragurilor de activare pentru a permite sistemului să identifice trăsături relevante în datele de intrare. În literatura de specialitate, metodele de antrenare a SNN-urilor pot fi clasificate în trei paradigme majore.

1.5.1 Conversia ANN în SNN

Această abordare presupune antrenarea prealabilă a unei rețele neuronale artificiale (ANN) convenționale, utilizând algoritmi standard precum *Backpropagation*, urmată de transferul și adaptarea ponderilor învățate către o arhitectură SNN echivalentă [KPPC20].

Procesul de conversie urmărește conservarea funcționalității și a performanței rețelei originale prin scalarea ponderilor și ajustarea acestora la intervalul dinamic specific modelului de neuron cu impulsuri ales. Deși această metodă oferă rezultate competitive și permite utilizarea infrastructurii software existente pentru *Deep Learning*, ea prezintă limitări critice:

- Necesită resurse computaționale ridicate pentru etapa inițială de antrenare a ANN-ului.
- Implică procese post-conversie complexe, precum normalizarea ponderilor și echilibrarea pragurilor (*threshold balancing*).
- Nu exploatează pe deplin avantajele temporale native ale SNN-urilor în timpul procesului de învățare.

1.5.2 Spiking Back-propagation

O altă metodă supervizată constă în adaptarea directă a algoritmului *Backpropagation* pentru domeniul impulsurilor [Gru23]. Această tehnică optimizează ponderile prin calcularea unei funcții de cost bazată pe eroarea dintre ieșirea dorită și cea prezisă, eroarea fiind ulterior propagată de la stratul de ieșire către straturile anterioare.

Principala provocare a acestei metode rezidă în faptul că funcția de activare a unui neuron *spiking* (funcția Heaviside) este discontinuă și, prin urmare, nederivabilă [LDP16]. Acest aspect împiedică calculul direct al gradientului necesar optimizării. Pentru a depăși acest obstacol, cercetările recente propun:

- Utilizarea unor **gradienți surogat** (*surrogate gradients*) care aproximează derivata impulsului în timpul etapei de *backward pass* [TGK+19].
- Tratarea potențialului de membrană ca pe un semnal continuu și derivabil [LDP16].

Deși metoda este eficientă pentru obținerea unei precizii ridicate, ea rămâne costisitoare din punct de vedere computațional și necesită volume mari de date etichetate, ceea ce limitează aplicabilitatea sa în sistemele de tip *on-chip learning* cu resurse reduse [LPSR18].

1.5.3 Învățarea de tip Hebbian

Plasticitatea de tip Hebbian constituie un principiu fundamental în neuroștiințe, stipulând că intensitatea unei sinapse între doi neuroni este determinată de corelația activităților acestora. Conform celebrului postulat al lui Donald Hebb: „neuronii care se activează împreună, se conectează împreună" (*neurons that fire together, wire together*) [Heb02]. În domeniul SNN, învățarea Hebbiană este utilizată pe scară largă pentru antrenarea rețelelor în regim supervizat [LXJL22], nesupervizat [KGTM18] sau auto-supervizat [HZ23].

Mecanismul de bază presupune modificarea greutății sinaptice în funcție de corelația dintre activitatea pre-sinaptică (intrarea) și cea post-sinaptică (ieșirea). Deși există mai multe reguli de plasticitate, precum *Spike-Driven Synaptic Plasticity* (SDSP), majoritatea aplicațiilor contemporane se bazează pe **Spike Timing-Dependent Plasticity (STDP)**, datorită robusteții sale în procesarea semnalelor temporale.

1.5.4 Spike Timing-Dependent Plasticity (STDP)

STDP este o regulă de învățare cu înaltă plauzibilitate biologică [SMA00] ce ajustează ponderile sinaptice pe baza diferenței temporale relative dintre impulsurile pre-sinaptice și cele post-sinaptice. Această metodă este esențială pentru implementările pe hardware neuromorfic cu consum redus, fiind capabilă să extragă trăsături complexe fără necesitatea unei supervizări externe.

Principiul de funcționare al STDP se bazează pe două fenomene opuse, așa cum este observat experimental în mod sistematic în [BP98]:

- **Potențarea pe termen lung (Long-Term Potentiation - LTP):** [BL73] Dacă impulsul pre-sinaptic are loc cu puțin timp înaintea impulsului post-sinaptic, sinapsa este întărită. Acest lucru sugerează că intrarea a contribuit direct la activarea neuronului.

- **Depresia pe termen lung (Long-Term Depression - LTD):** [DB92] Dacă impulsul post-sinaptic are loc înaintea celui pre-sinaptic, sinapsa este slăbită. Aceasta indică faptul că semnalul de intrare nu a avut un rol cauzal în declanșarea impulsului de ieșire.

Această dinamică permite rețelei să prioritizeze conexiunile care transportă informație relevantă pentru modelul identificat, ignorând în același timp zgomotul sau semnalele necorelate temporal.

Variante ale regulii STDP

În funcție de complexitatea dorită și de constrângerile hardware, STDP poate fi implementată prin mai multe modele matematice:

1. STDP Biologic: Este forma de bază care utilizează o descreștere exponențială a actualizării ponderii în funcție de distanța temporală dintre impulsuri [BP98]. Formalizarea matematică este dată de ecuația (1.9):

$$\Delta w = \begin{cases} \eta_w e^{-\frac{t_{post}-t_{pre}}{\tau_{STDP}}}, & \text{dacă } t_{pre} \leq t_{post} \\ -\eta_w e^{-\frac{t_{pre}-t_{post}}{\tau_{STDP}}}, & \text{în caz contrar} \end{cases} \tag{1.9}$$

unde η_w reprezintă rata de învățare, iar τ_{STDP} este constanta de timp ce definește fereastra de învățare.

2. STDP Aditiv (Add-STDP): O variantă simplificată unde modificarea ponderii depinde doar de semnul diferenței temporale, nu și de magnitudine. Deoarece poate conduce la instabilitate, aceasta necesită limite stricte de control (*hard boundaries*), rezultând adesea în ponderi binare (0 sau 1) [SMA00]:

$$\Delta w = \begin{cases} \eta_{w+}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-}, & \text{în caz contrar} \end{cases} \tag{1.10}$$

3. STDP Multiplicativ (Mult-STDP): Această variantă introduce dependența de valoarea curentă a ponderii, prevenind saturarea și oferind o distribuție mai fină a greutăților sinaptice, comparativ cu modelul aditiv [vRBT00]. Actualizarea este reglată astfel încât ponderile mici să poată crește mai rapid, în timp ce ponderile mari sunt limitate superior:

$$\Delta w = \begin{cases} \eta_{w+} e^{-\beta \frac{w-w_{min}}{w_{max}-w_{min}}}, & \text{dacă } \delta t \leq 0 \\ -\eta_{w-} e^{-\beta \frac{w_{max}-w}{w_{max}-w_{min}}}, & \text{în caz contrar} \end{cases} \tag{1.11}$$

1.5.5 Inhibiția

Inhibiția reprezintă procesul de diminuare sau suprimare a activității neuronale, fiind un mecanism fundamental pentru reglarea dinamicii corticale [SM07]. Într-o rețea neuronală cu impulsuri (SNN), inhibiția joacă un rol vital în menținerea echilibrului dintre mecanismele excitatorii și cele inhibitorii, permițând un control precis asupra fluxului de informație și a momentelor de declanșare a impulsurilor [ZZL22].

Cea mai utilizată metodă de inhibiție în faza de antrenare este mecanismul *Winner-Take-All* (WTA) [QBG11, Che17]. Conform acestei reguli, primul neuron dintr-un

strat care generează un impuls este declarat „câștigător" și transmite imediat un semnal inhibitor către ceilalți neuroni din același strat pentru a le bloca activitatea. Această strategie este deosebit de eficientă în învățarea nesupervizată bazată pe STDP, deoarece introduce o competiție între neuroni [FMT18]. Prin împiedicarea mai multor neuroni să răspundă simultan la același stimul, rețeaua este forțată să învețe trăsături distincte și mai puțin corelate, optimizând astfel capacitatea de reprezentare a sistemului.

1.5.6 Homeostazia

Deși mecanismul *Winner-Take-All* (WTA) este esențial pentru diversificarea procesului de învățare, acesta poate introduce un dezechilibru în care un număr restrâns de neuroni devin dominanți. Acești neuroni „câștigători" tind să se declanșeze repetat, monopolizând procesul de adaptare a ponderilor, în timp ce restul unităților rămân inactive. Pentru a evita această saturație și a menține stabilitatea sistemului, este necesară implementarea unor mecanisme de homeostazie prin adaptarea pragului de declanșare (*threshold adaptation*).

În cadrul acestei arhitecturi, se utilizează două metode complementare de adaptare a pragului. Prima și cea mai importantă este **Leaky Adaptive Threshold (LAT)** [DC15b], utilizată pentru a asigura o competiție echitabilă între neuroni. Cea de-a doua este metoda **adaptării pragului bazată pe un timp țintă** (*target timestamp*) [FTB+19], utilizată în conjuncție cu LAT în modelele de tip CSNN de ultimă generație [Fal19]. Aceasta din urmă forțează neuronii să tindă spre un moment de declanșare predefinit $\hat{t}$, permițând modelului să se specializeze pe anumite tipare temporale, contribuind în același timp la homeostazia rețelei.

Pragurile tuturor neuronilor dintr-un strat (atât pentru cel câștigător, cât și pentru cei care au primit semnal de inhibiție) sunt actualizate la fiecare declanșare a unui impuls sau la recepția unui semnal inhibitor. Din punct de vedere matematic, modificarea pragului se bazează pe două componente, Δ_{th}^1 și Δ_{th}^2, după cum urmează:

$$\Delta_{th}^1 = -\eta_{th}(t - \hat{t}) \tag{1.12}$$

unde Equation (1.12) reprezintă adaptarea bazată pe timpul țintă, în care t este momentul declanșării impulsului, $\hat{t}$ este timpul țintă dorit, iar η_{th} este rata de învățare a pragului. Componenta LAT este definită în Equation (1.13):

$$\Delta_{th}^2 = \begin{cases} \eta_{th}, & \text{dacă } t_i = \min\{t_0, \ldots, t_N\} \\ -\frac{\eta_{th}}{ld(n)}, & \text{în caz contrar} \end{cases} \tag{1.13}$$

unde t_i reprezintă timpul de declanșare al neuronului i, $\min\{t_0, \ldots, t_N\}$ corespunde neuronului care a generat primul impuls (câștigătorul), iar $ld(n)$ reprezintă numărul de neuroni aflați în competiție în stratul respectiv. În final, actualizarea potențialului pragului de membrană V_{th} se realizează conform Equation (1.14):

$$V_{th}(t) = \max(th_{min}, V_{th}(t-1) + \Delta^1_{th} + \Delta^2_{th}) \tag{1.14}$$

unde th_{min} reprezintă valoarea minimă admisă a pragului [Fal19], asigurând astfel că sensibilitatea neuronului nu crește peste o limită care ar putea genera activitate spontană cauzată de zgomot. Această abordare combinată asigură faptul că latența impulsurilor converge către valoarea țintă, menținând în același timp un echilibru între activitatea tuturor celor N neuroni din strat.

1.6 Arhitecturi Spiking Convolutional Neural Networks (CSNN)

Arhitecturile Spiking Convolutional Neural Networks (CSNN) combină capacitatea de extragere a trăsăturilor spațiale cu eficiența energetică a neuronilor de tip spiking. Implementarea unui strat de convoluție în acest context presupune o metodologie structurată în două etape distincte: etapa de învățare a filtrelor și etapa de generare a hărților de trăsături (**feature maps**), după cum urmează:

1. **Etapa de învățare (Learning Phase):** În această fază, fiecare nucleu de convoluție (**kernel**) este asociat unui neuron specific (sau unui grup restrâns de neuroni). Scopul acestei etape este antrenarea ponderilor sinaptice W, adesea utilizând regula STDP, pentru a specializa nucleul în detectarea unui anumit tipar vizual.

2. **Etapa de testare/inferență (Inference Phase):** Odată ce ponderile au fost învățate, acestea sunt „înghețate" și replicate pe întreaga suprafață a intrării. În această etapă, se asociază neuroni virtuali pentru fiecare element din hărțile de trăsături rezultate. Esențial este faptul că toate unitățile dintr-o hartă de trăsături partajează același nucleu învățat anterior, asigurând invarianța la translație.

1.6.1 Convoluția în domeniul impulsurilor (2D-CSNN)

Generarea hărților de trăsături în etapa de inferență se realizează prin aplicarea glisantă a filtrului, unde dinamica potențialului de membrană pentru un neuron de tip *Integrate-and-Fire* (IF) de la coordonatele (x, y) este calculată conform ecuației (1.15):

$$V_{m_{x,y,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),k_i,k} \cdot f_s(t - t_i) \tag{1.15}$$

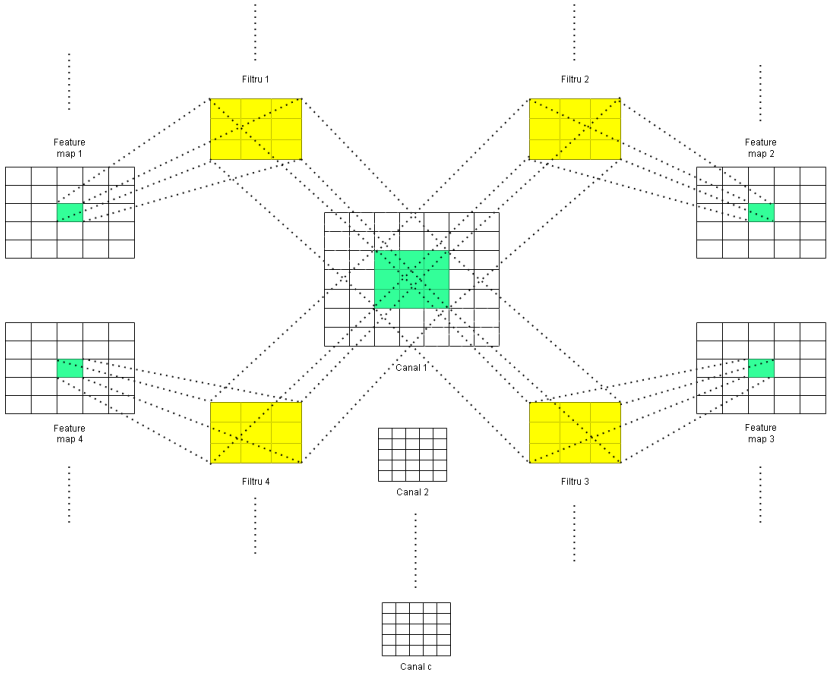


Figura 1.4: Reprezentarea procesului de convoluție 2D. În faza de testare, nucleul învățat este aplicat pe toată imaginea pentru a genera harta de trăsături.

1.6.2 3D-CSNN pentru procesare video

Extensia către 3D-CSNN urmează aceeași logică bimodală (învățare vs. testare), dar extinde domeniul de aplicare în dimensiunea temporală. Această arhitectură permite extragerea directă a trăsăturilor spatio-temporale, eliminând necesitatea unor algoritmi externi de calcul al fluxului optic [HS81].

În etapa de inferență 3D, nucleul glisant integrează informația dintr-un volum de intrare (lățime, înălțime și adâncime temporală). Toți neuronii dintr-un volum de ieșire k utilizează același set de ponderi 3D învățate, calculând potențialul de membrană la momentul t conform formulei:

$$V_{m_{x,y,z,k}}(t) = \sum_{i \in E} W_{n(x_i),j(y_i),h(z_i),k_i,k} \cdot f_s(t - t_i) \tag{1.16}$$

unde $h(z_i)$ mapează coordonata temporală a impulsului în fereastra nucleului 3D. Acest mecanism permite neuronilor să devină selectivi la schimbări dinamice, simulând comportamentul neuronilor din cortexul vizual primar care răspund la direcții specifice de mișcare în câmpul receptiv (Figura 1.5).

1.7 Algoritmi de preprocesare și detecție facială

Pentru a asigura o rată ridicată de recunoaștere a emoțiilor, sistemul utilizează o etapă riguroasă de preprocesare. Aceasta are rolul de a extrage doar informația

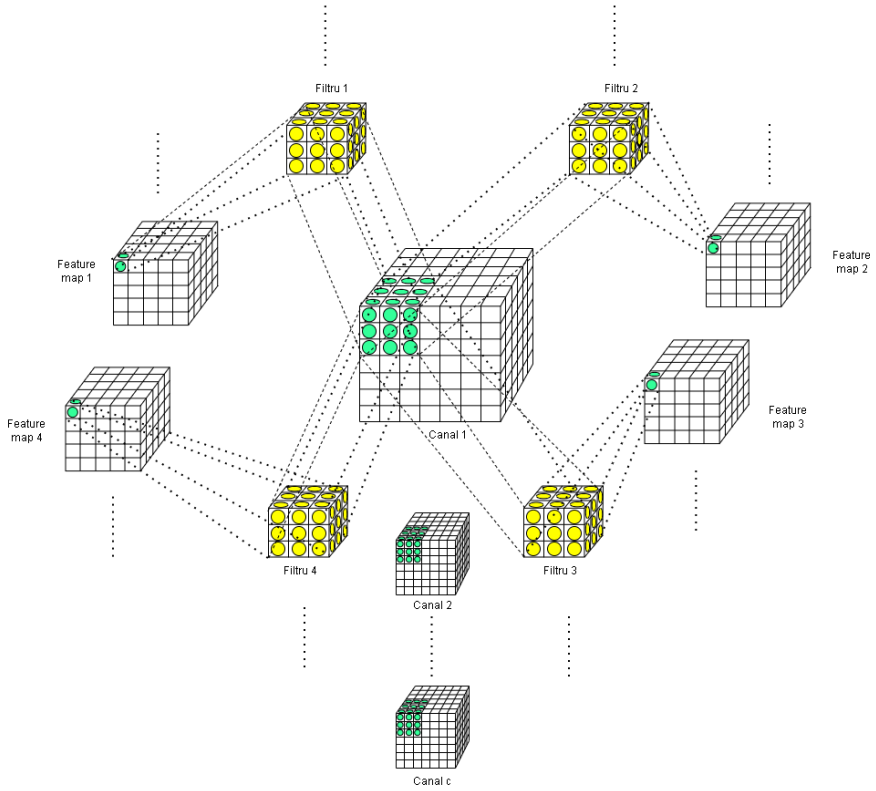


Figura 1.5: Operația de convoluție 3D. Aceasta procesează „cuburi" de evenimente, unde aceeași matrice de ponderi W este utilizată pentru a scana întregul flux video.

relevantă (fața subiectului) și de a menține alinierea acesteia pe parcursul întregii secvențe video, proces adaptat conform metodologiei descrise în [TCKPS20].

1.7.1 Detecția feței prin algoritmul Viola-Jones

În primul cadru al secvenței video, sistemul nu deține informații despre locația subiectului. Pentru inițializare, se utilizează algoritmul **Viola-Jones** [VJ01], un standard în domeniu datorită eficienței sale computaționale.

Algoritmul funcționează prin scanarea imaginii cu ferestre glisante de dimensiuni variabile, căutând trăsături simple numite *Haar-like features*. Acestea se bazează pe diferențele de intensitate dintre regiuni adiacente (de exemplu, zona ochilor este de regulă mai întunecată decât pometii, iar puntea nasului este mai luminoasă decât zonele laterale). Rezultatul acestei etape este definirea unei **Regiuni de Interes (ROI)** sub forma unui dreptunghi care încadrează fața detectată.

1.7.2 Extragerea și urmărirea punctelor de interes

Odată ce ROI a fost stabilit, procesarea întregii suprafețe a pielii în fiecare cadru ar fi ineficientă și predispusă la erori din cauza texturii uniforme. Din acest motiv, se trece la identificarea unor puncte de „ancoră".

Algoritmul Min-Eigenvalue: Pentru a selecta cele mai robuste puncte pentru urmărire, se utilizează metoda *Min-Eigenvalue* (cunoscută și sub numele de detectorul Shi-Tomasi [ST94]). Acest algoritm identifică colțuri — puncte în care intensitatea pixelilor se schimbă brusc în două direcții ortogonale. Sunt selectate astfel puncte unice precum colțurile ochilor, vârful nasului sau extremitățile gurii, acestea fiind ușor de „agățat" și urmărit în cadrele subsecvente.

Urmărirea prin algoritmul KLT (Kanade-Lucas-Tomasi): În loc să ruleze detecția Viola-Jones în fiecare cadru (proces costisitor), sistemul utilizează algoritmul KLT [LK81]. Acesta reprezintă „motorul" de mișcare al preprocesării, analizând fluxul optic (*optical flow*) pentru a determina unde s-au mutat punctele ancoră din cadrul curent în cadrul următor. KLT este optimizat pentru „micro-mișcări", fiind mult mai rapid decât o detecție de la zero.

1.7.3 Transformări geometrice și alinierea feței

Deoarece subiectul se poate mișca liber (rotații ale capului, aplecări sau schimbări de scară), simpla urmărire a punctelor nu este suficientă pentru a menține un cadru stabil pentru SNN.

Actualizarea ROI prin Geometric Transform: Dacă punctele ancoră detectate la pasul anterior și-au schimbat coordonatele, dreptunghiul inițial trebuie actualizat, acest lucru având loc cu ajutorul matricii de transformare M (translație, rotație și scalare) care descrie mișcarea medie a tuturor punctelor. Această matrice este aplicată întregului poligon ROI, permițând „ramei" feței să sară exact pe noua poziție.

1.8 Metode de eșantionare a datelor de intrare (Sampling)

În cadrul arhitecturilor de tip Convolutional Spiking Neural Networks (CSNN), procesul de eșantionare (sampling) joacă un rol critic în eficiența procesării informației vizuale. Spre deosebire de rețelele neurale convoluționale clasice, unde întreaga imagine este procesată uniform, simulatoarele de SNN pot utiliza strategii de eșantionare pentru a selecta ferestre locale (patch-uri) care urmează să fie convertite în trenuri de impulsuri (spikes).

Această abordare este inspirată din mecanismele biologice de atenție vizuală, permițând modelului să se concentreze pe regiunile cele mai informative ale imaginii, reducând astfel costul computațional și zgomotul de fundal. În cadrul acestui

simulator, au fost implementate trei strategii principale de eșantionare, atât 2D, cât și 3D.

1.8.1 Eșantionarea Spațială (2D)

Eșantionarea Aleatorie

Metoda de eșantionare aleatorie reprezintă abordarea de bază, în care locația ferestrei de filtrare este determinată în mod stohastic pe întreaga suprafață a imaginii de intrare. Coordonatele colțului stânga-sus al ferestrei, (x, y), sunt extrase dintr-o distribuție uniformă, respectând constrângerile dimensionale ale imaginii:

$$x \sim \mathcal{U}(0, W - w_f), \quad y \sim \mathcal{U}(0, H - h_f) \tag{1.17}$$

unde W, H reprezintă lățimea și înălțimea imaginii, iar w_f, h_f dimensiunile filtrului. Deși această metodă asigură o acoperire imparțială a spațiului de intrare, ea este ineficientă în sarcini de recunoaștere a emoțiilor faciale, deoarece o mare parte din eșantioane pot proveni din zone irelevante (fundal).

Eșantionarea bazată pe Regiunea Facială

Pentru a restrânge spațiul de căutare la zona de interes (Region of Interest - ROI), se utilizează casete de delimitare (bounding boxes) detectate în prealabil. Această metodă asigură faptul că ferestrele eșantionate se suprapun cu zona feței.

Procesul de selecție a coordonatelor este limitat de limitele minime și maxime ale casetei de delimitare, asigurând astfel că eșantionul rezultat va conține informație relevantă pentru sarcina de clasificare a emoțiilor.

Eșantionarea bazată pe Puncte de Reper

Cea mai avansată metodă implementată este eșantionarea bazată pe puncte de reper (landmarks). Această strategie se fundamentează pe ipoteza că emoțiile faciale sunt cel mai bine reprezentate de mișcările micro-expresiilor din zonele ochilor, nasului și gurii.

Logica de funcționare presupune parcurgerea următoarelor etape:

1. **Selecția punctului cheie:** Se alege în mod aleatoriu un punct de reper dintr-un set predefinit (corespunzător indicilor 17–67 în formatul standard de 68 de puncte, după cum este ilustrat în Figura 1.6), care acoperă trăsăturile interne ale feței.

2. **Centrarea ferestrei:** Coordonatele ferestrei sunt calculate astfel încât punctul de reper selectat să fie poziționat în centrul patch-ului de $w_f \times h_f$.

3. **Introducerea zgomotului (Jitter):** Pentru a crește robustețea modelului și a evita supra-învățarea pe poziții fixe, se adaugă un factor de variație aleatorie (jitter) în intervalul $[-2, 2]$ pixeli prin aplicarea distribuției uniforme.

4. **Constrângere (Clamping):** Se aplică o funcție de *clamping* pentru a asigura că fereastra rezultată nu depășește limitele fizice ale imaginii.

Această selecție a indicilor nu este arbitrară, ci urmează schema de adnotare propusă în setul de date 300-W [STZP13], utilizată pe scară largă în bibliotecile de procesare a feței (precum Dlib sau MediaPipe), detalierea fiecărei regiuni faciale fiind prezentată în Tabelul 1.1. Intervalul $[17, 67]$ a fost ales pentru a exclude conturul feței (jawline) și a maximiza probabilitatea de a eșantiona trăsături care definesc starea emoțională (ochi, sprâncene, gură).

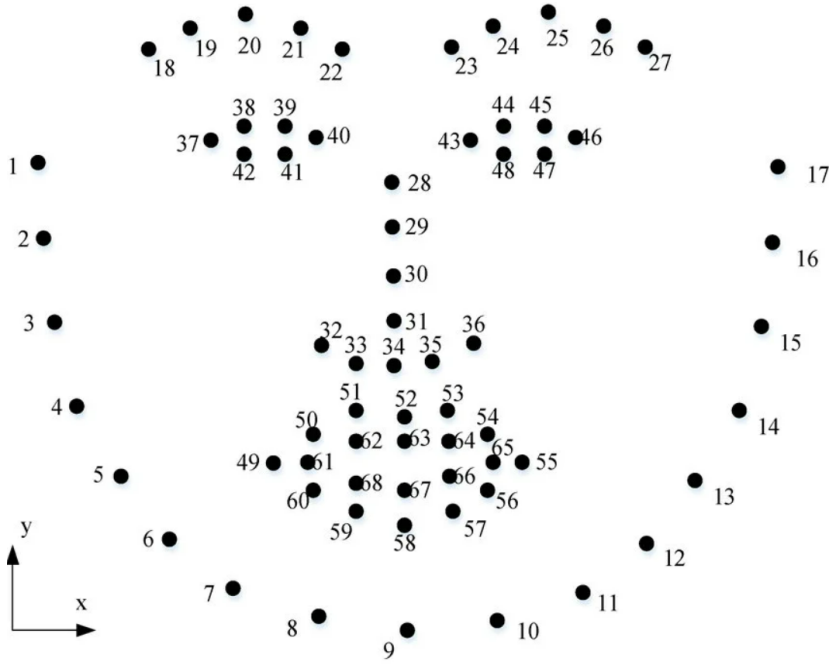


Figura 1.6: Reprezentarea celor 68 de puncte de reper faciale (landmarks) conform standardului 300-W [LWG+19].

Prin utilizarea acestor repere, procesul de eșantionare imită modul în care sistemul vizual uman prioritizează punctele de înaltă frecvență informațională pentru a recunoaște expresiile faciale complexe.

1.8.2 Eșantionarea Spatio-Temporală (3D)

În cazul procesării secvențelor video pentru recunoașterea emoțiilor faciale, modelul trebuie să capteze nu doar trăsăturile statice, ci și dinamica temporală a expresiilor. Astfel, eșantionarea 3D extinde conceptele anterioare prin adăugarea unei a treia dimensiuni: timpul (t). Rezultatul procesului de eșantionare este un volum de

Tabela 1.1: Distribuția punctelor de reper faciale în formatul standard de 68 de puncte.

Regiune Facială	**Indici (0-indexed)**
Contur Mandibulă	0 – 16
Sprânceana Dreaptă	17 – 21
Sprânceana Stângă	22 – 26
Puntea Nazală	27 – 30
Baza Nasului	31 – 35
Ochiul Drept	36 – 41
Ochiul Stâng	42 – 47
Contur Exterior Gură	48 – 59
Contur Interior Gură	60 – 67

date definit de coordonatele (x, y, t), unde t reprezintă punctul de start al ferestrei temporale.

Eșantionarea Aleatorie

Această metodă generează în mod stohastic toate cele trei coordonate. Fereastra temporală este selectată dintr-o distribuție uniformă, asigurând că volumul extras se încadrează în durata totală a secvenței video (T):

$$t \sim \mathcal{U}(0, T - t_f) \tag{1.18}$$

unde t_f reprezintă dimensiunea filtrului pe axa temporală.

Strategii Faciale bazate pe Bounding Box

Pentru a gestiona mișcarea feței de-a lungul timpului, au fost implementate două strategii de restrângere a spațiului de căutare:

- **Per Frame:** Această metodă este dinamică. Pentru fiecare fereastră temporală selectată, se calculează o regiune de reuniune (*Union BBox*) care cuprinde toate casetele de delimitare ale cadrelor din intervalul $[t, t + t_f]$. Aceasta garantează că, indiferent de mișcarea capului în acea fracțiune de secundă, eșantionul va conține fața subiectului.
- **Per Video:** Reprezintă o abordare mai conservatoare, unde se calculează un singur *Union BBox* pentru întreaga durată a videoclipului. Este utilă în scenarii cu mișcare redusă, oferind o stabilitate spațială mai mare eșantioanelor extrase.

Eșantionarea bazată pe Landmark-uri

Logica acestei metode combină precizia punctelor de reper cu dimensiunea temporală. Procesul decurge astfel:

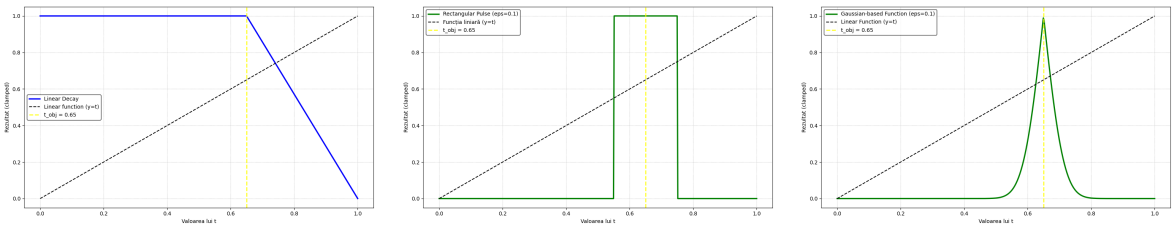
1. Se alege aleatoriu momentul de start t.
2. Se accesează datele despre punctele de reper specifice cadrului t.
3. Se selectează un punct de reper facial (din intervalul de indici $17 - 67$) și se centrează fereastra spațială (x, y) în jurul acestuia.

Această eșantionare permite rețelei CSNN să analizeze evoluția temporală a unei trăsături faciale specifice (de exemplu, curbura gurii în timpul unui zâmbet), eliminând necesitatea procesării întregului cadru și focalizând activitatea neuronală (spikes) pe zonele cu relevanță semantică maximă.

1.9 Metode de Decodificare a Semnalelor Neuronale

În cadrul arhitecturilor de tip Convolutional Spiking Neural Networks (CSNN), etapa de decodificare reprezintă puntea de legătură între activitatea neuronală discretă (spike-uri) și interpretarea acestora ca probabilități pentru expresiile faciale. Deoarece informația în CSNN este codificată temporal, am implementat trei strategii distincte de ponderare a activității neuronale pentru a evalua relevanța spike-urilor în raport cu un moment de referință t_{obj} setat la antrenare, ce reprezintă timpul obiectiv la care să aibă loc spike-urile.

Comportamentul vizual al acestor funcții de decodificare este prezentat în Figura 1.7.



(a) Linear Decay (b) Rectangular Pulse (c) Gaussian-based

Figura 1.7: Vizualizarea comparativă a funcțiilor de decodificare utilizate.

1.9.1 Decodificarea prin Declin Liniar (Linear Decay)

Această metodă, ilustrată în Figura 1.7a, aplică o descreștere constantă a importanței semnalului pe măsură ce timpul t avansează dincolo de pragul t_{obj}. Ea a fost selec-

tată pentru eficiența sa computațională și pentru capacitatea de a modela degradarea simplă a memoriei pe termen scurt a neuronilor, oferind o penalizare proporțională pentru spike-urile care apar târziu în fereastra de procesare a expresiei faciale.

1.9.2 Decodificarea prin Impuls Dreptunghiular (Rectangular Pulse)

Metoda impulsului dreptunghiular (Figura 1.7b) acționează ca un filtru binar de tip "fereastră". Aceasta atribuie o importanță maximă (1.0) tuturor spike-urilor care apar într-un interval strict de ± 0.1 față de centrul t_{obj}, ignorând complet restul activității. Această abordare este utilă pentru detectarea coincidențelor temporale rigide, eliminând zgomotul de fond și spike-urile izolate care nu se încadrează în dinamica temporală specifică unei anumite micro-expresii.

1.9.3 Decodificarea Gaussiană (Gaussian-based Importance)

Cea mai sofisticată metodă, bazată pe funcția de eroare (erf), generează o curbă de tip clopot centrată pe t_{obj} (Figura 1.7c). Spre deosebire de metodele anterioare, aceasta aproximează cel mai bine variabilitatea biologică a neuronilor și oferă o tranziție lină (smooth transition), fiind robustă la mici variații temporale.

1.10 Percepția vizuală bazată pe evenimente

1.10.1 Senzori de viziune dinamică (DVS)

1.10.2 Recunoașterea activităților umane (HAR) folosind SNN

Capitolul 2

Recunoașterea facială și procesarea semnalului vizual

2.1 Biometria facială: provocări și oportunități

2.2 Reprezentarea datelor sub formă de impulsuri (Spike Encoding)

2.2.1 Rate Coding vs. Temporal Coding

2.3 Problematica zgomotului de fundal în sistemele neuromorfice

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis

nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Capitolul 3

Preprocesarea imaginii și segmentarea Regiunii de Interes (ROI)

3.1 Importanța segmentării în eficientizarea CSNN

3.2 Algoritmi pentru detecția feței

3.3 Extragerea ROI și normalizarea pentru conversia în impulsuri

3.4 Reducerea activității neuronale redundante prin eliminarea fundalului

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis

nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

Capitolul 4

Proiectarea și implementarea sistemului de recunoaștere facială

4.1 Arhitectura sistemului propus (Pipeline-ul de procesare)

4.1.1 Simulatorul CSNN

4.2 Tehnologii și biblioteci utilizate

4.2.1 Biblioteci de procesare vizuală (OpenCV)

4.3 Configurarea straturilor convoluționale ale CSNN

4.4 Mecanismul de antrenare

Capitolul 5

Experimente și analiza rezultatelor

5.0.1 Analiza Arhitecturilor de Clasificare în CSNN

Alegerea stratului final de decizie într-o rețea neuronală convoluțională de tip spiking (CSNN) reprezintă un compromis între eficiența computațională, precizia clasificării și plauzibilitatea biologică. În această secțiune, analizăm cele două abordări fundamentale utilizate în literatura de specialitate.

Abordarea Hibridă: CSNN cu Clasificator SVM

Această arhitectură utilizează straturile convoluționale de tip spiking exclusiv pentru extracția de trăsături (feature extraction), în timp ce decizia finală este delegată unui algoritm Support Vector Machine (SVM).

Motivația utilizării:

- **Separabilitatea Datelor:** Trăsăturile extrase prin procese nesupervizate (precum STDP - Spike-Timing-Dependent Plasticity) nu sunt întotdeauna liniar separabile. SVM utilizează un kernel pentru a proiecta aceste date într-un spațiu de dimensiune superioară unde se poate determina un hiperplan optim de separare:
$$f(x) = \text{sgn}\left(\sum_{i=1}^{n} \alpha_i y_i K(x_i, x) + b\right)$$
- **Robustete pe Seturi de Date Reduse:** SVM implementează principiul minimizării riscului structural, fiind mult mai eficient decât un strat neuronal atunci când numărul de mostre de antrenament pentru anumite emoții este limitat.
- **Decuplarea Învățării:** Permite antrenarea rapidă a clasificatorului fără a fi necesară recalcularea întregii rețele spiking, facilitând experimentarea cu diferite meta-argumente de clasificare.

Abordarea End-to-End: CSNN cu Straturi Fully Connected Spiking

În această configurație, rețeaua rămâne integral în domeniul spiking, unde ultimele straturi sunt compuse din neuroni complet conectați (FC) care emit impulsuri pentru a reprezenta apartenența la o clasă.

Motivația utilizării:

- **Eficiență Energetică și Hardware Neuromorfic:** O rețea pură poate fi implementată direct pe cipuri neuromorfice (ex. Intel Loihi, IBM NorthPole). Introducerea unui SVM ar necesita conversia semnalului înapoi în domeniul analogic/numeric, anulând avantajele de consum redus ale SNN.
- **Păstrarea Informației Temporale:** Spre deosebire de SVM, care procesează adesea rate de activare (spike rates) statice, straturile FC spiking pot utiliza codificarea temporală (Time-to-First-Spike). Dinamica membranei neuronale este descrisă de ecuația:

$$\tau_m \frac{dv}{dt} = -(v - v_{rest}) + R \cdot I(t)$$

  unde decizia este influențată direct de momentul precis al sosirii impulsurilor.
- **Antrenament prin Surrogate Gradients:** Utilizarea algoritmilor de backpropagation prin timp (BPTT) cu gradienți substituiți permite optimizarea întregului lanț de procesare, de la pixeli până la eticheta emoției, asigurând o coerență matematică a întregului sistem.

5.1 Setul de date utilizat

5.2 Metodologia de testare a modelului

5.2.1 Cazul A: Recunoaștere pe imagini brute (cu fundal)

5.2.2 Cazul B: Recunoaștere pe imagini cu segmentare ROI

5.3 Metrici de performanță

5.4 Analiza impactului zgomotului vizual asupra preciziei modelului

Capitolul 6

Numele aplicației

Capitolul 7

Colectarea cerințelor

În cadrul acestui capitol sunt detaliate funcționalitățile sistemului Neuro-Guardian, interacțiunile dintre utilizatori și platformă, precum și constrângerile tehnice sub care funcționează soluția propusă.

7.1 Actorii sistemului

Actorii reprezintă entitățile externe care interacționează cu sistemul. În cadrul platformei Neuro-Guardian, am identificat următoarele categorii:

- **Angajat Spital:** Actorul generic care include funcționalitățile de bază de autentificare (Înregistrare, Autentificare, Deconectare).
- **Administrator:** Moștenește trăsăturile angajatului și are responsabilitatea gestionării fluxului administrativ al pacienților (internări, externări și alocarea resurselor/paturilor).
- **Doctor:** Actorul principal în procesul de îngrijire, responsabil de preluarea pacienților și monitorizarea stării lor clinice și emoționale în timp real.
- **Camera Web (Sistem Extern):** Furnizează fluxul de date video necesar procesului de captare a cadrelor.
- **CSNN - Convolutional Spiking Neural Network (Sistem Extern):** Modelul de inteligență artificială care procesează cadrele recepționate și returnează predicțiile emoționale.

7.2 Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare, prezentată în Figura 7.1, oferă o imagine de ansamblu asupra modului în care funcționalitățile sunt distribuite între actori.

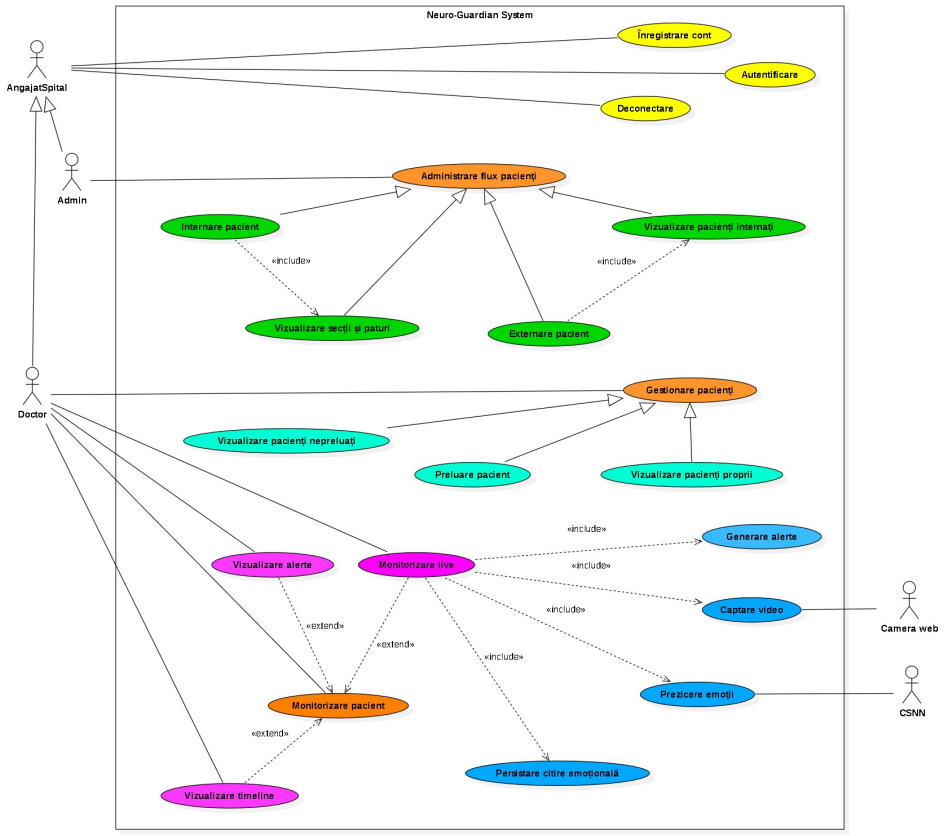


Figura 7.1: Diagrama cazurilor de utilizare - Sistemul Neuro-Guardian

7.3 Descrierea detaliată a unui caz de utilizare

În această secțiune este detaliat fluxul principal de interacțiune pentru cazul de utilizare **Gestionare Pacienți**, reprezentat grafic în Figura 7.1. Acest caz de utilizare este central pentru activitatea clinică a medicului, asigurând tranziția între selecția pacienților și monitorizarea activă.

7.4 Cerințe nefuncționale (NFR)

Cerințele nefuncționale definesc criteriile de calitate și constrângerile tehnice ale sistemului.

7.4.1 Securitate și control acces

- **NFR-S1:** Toate operațiile sensibile necesită autentificare pe bază de Bearer JWT.
- **NFR-S2:** Autorizare strictă pe rol (ADMIN vs DOCTOR); medicii pot vizualiza doar pacienții care le sunt asignați.

Elemente de identificare	
Nume	Gestionare pacienți
Actori	Doctor
Descriere	Procesul prin care un medic vizualizează lista pacienților din secție, preia responsabilitatea pentru un pacient nou sau comută între pacienții deja asignați pentru a vizualiza datele biometrice și emoționale.
Condiții de context	
Precondiții	1. Medicul este autentificat în sistem. 2. Pacienții sunt deja internați în secție (via Admin). 3. Conexiunea WebSocket cu modelul AI este activă.
Postcondiții	Interfața reflectă starea în timp real a pacientului selectat, iar baza de date actualizează relația medic-pacient.
Scenariu Principal de Succes (MSS)	
Flux normal	1. Medicul accesează dashboard-ul; sistemul afișează implicit datele ultimului pacient preluat. 2. Medicul acționează butonul „Show panel" pentru a extinde panoul de selecție. 3. Sistemul interoghează baza de date și afișează două liste: „Available Patients" și „My Patients". 4. Medicul identifică un pacient nepreluat și selectează opțiunea „Take patient". 5. Sistemul realizează maparea medic-pacient și trimite medicul către pagina destinată pacientului respectiv 6. Medicul selectează un alt pacient din lista „My Patients" pentru a verifica evoluția acestuia. 7. Sistemul comută contextul de monitorizare, afișând informațiile specifice noului pacient selectat. 8. Medicul acționează „Hide panel" pentru a maximiza spațiul de vizualizare a detaliilor pacientului. 9. Sistemul ascunde panoul lateral și trece în modul de monitorizare imersivă.
Scenarii alternative	
Alt. 1	**Medicul nu are pacienți în responsabilitate:** 1a. Sistemul afișează un mesaj de informare și prezintă doar lista pacienților nepreluați. 1b. Medicul efectuează preluarea primului pacient (se revine la pasul 5 din MSS).
Alt. 2	**Inexistența pacienților nepreluați:** 3a. Lista „Available patients" apare goală; medicul poate gestiona doar portofoliul actual de pacienți (se revine la pasul 6 din MSS).
Excepții	
Excepție 1	**Pacientul este externat simultan:** Sistemul blochează preluarea și notifică medicul că datele nu mai sunt disponibile.
Cerințe de calitate	
Performanță	Timpul de comutare între doi pacienți nu trebuie să depășească 1 secundă pentru datele textuale.
Fiabilitate	Persistarea asocierii medic-pacient trebuie să fie confirmată tranzacțional în baza de date.

Tabela 7.1: Descrierea detaliată a scenariului de utilizare: Gestionare pacienți

- **NFR-S3:** Parolele sunt stocate exclusiv în format hashed folosind algoritmul *bcrypt*.
- **NFR-S4:** Token-urile au timp de expirare implementat, iar refresh token-urile sunt rotite automat și invalidate la logout.
- **NFR-S5:** Validarea riguroasă a input-ului la frontierele API (câmpuri obligatorii, formate de timestamp valide).

7.4.2 Confidențialitate și protecția datelor

- **NFR-P1:** Datele pacienților sunt accesibile doar personalului medical autorizat direct.
- **NFR-P2:** Minimizarea expunerii datelor; sistemul nu afișează informații care nu sunt necesare contextului clinic curent.
- **NFR-P3:** Comunicația între client și server se realizează prin protocoale criptate HTTPS/WSS (TLS).

7.4.3 Performanță și timp real

- **NFR-F1:** Afișarea emoțiilor se realizează aproape în timp real prin WebSocket, cu o latență minimă, în timp ce persistarea în baza de date se face periodic.
- **NFR-F2:** Limitarea backlog-ului de cadre video procesate pentru a preveni consumul excesiv de memorie RAM pe client.
- **NFR-F3:** Dashboard-ul rămâne responsiv și funcțional chiar și în perioadele de indisponibilitate a motorului de analiză AI.

7.4.4 Fiabilitate și disponibilitate

- **NFR-R1:** Mecanism de *fallback*: dacă InfluxDB este indisponibil, sistemul utilizează baza de date principală (Postgres) sau date sintetice pentru continuitate.
- **NFR-R2:** Interfața semnalează vizual deconectarea modelului AI, permițând utilizatorului să continue alte activități administrative.
- **NFR-R3:** Operațiile critice (Check-in/Check-out) sunt tratate tranzacțional pentru a menține consistența între starea patului și cea a pacientului.

7.4.5 Uzabilitate și Mentenabilitate

- **NFR-U1:** Interfața oferă feedback vizual clar pentru stările de încărcare sau eroare (Loading/Error states).
- **NFR-M1:** Arhitectură modulară bazată pe TypeScript, cu separare clară între controller, rute și servicii.
- **NFR-M3:** Aplicația este cross-platform (Windows/Linux) utilizând stack-ul Node.js și Vite.

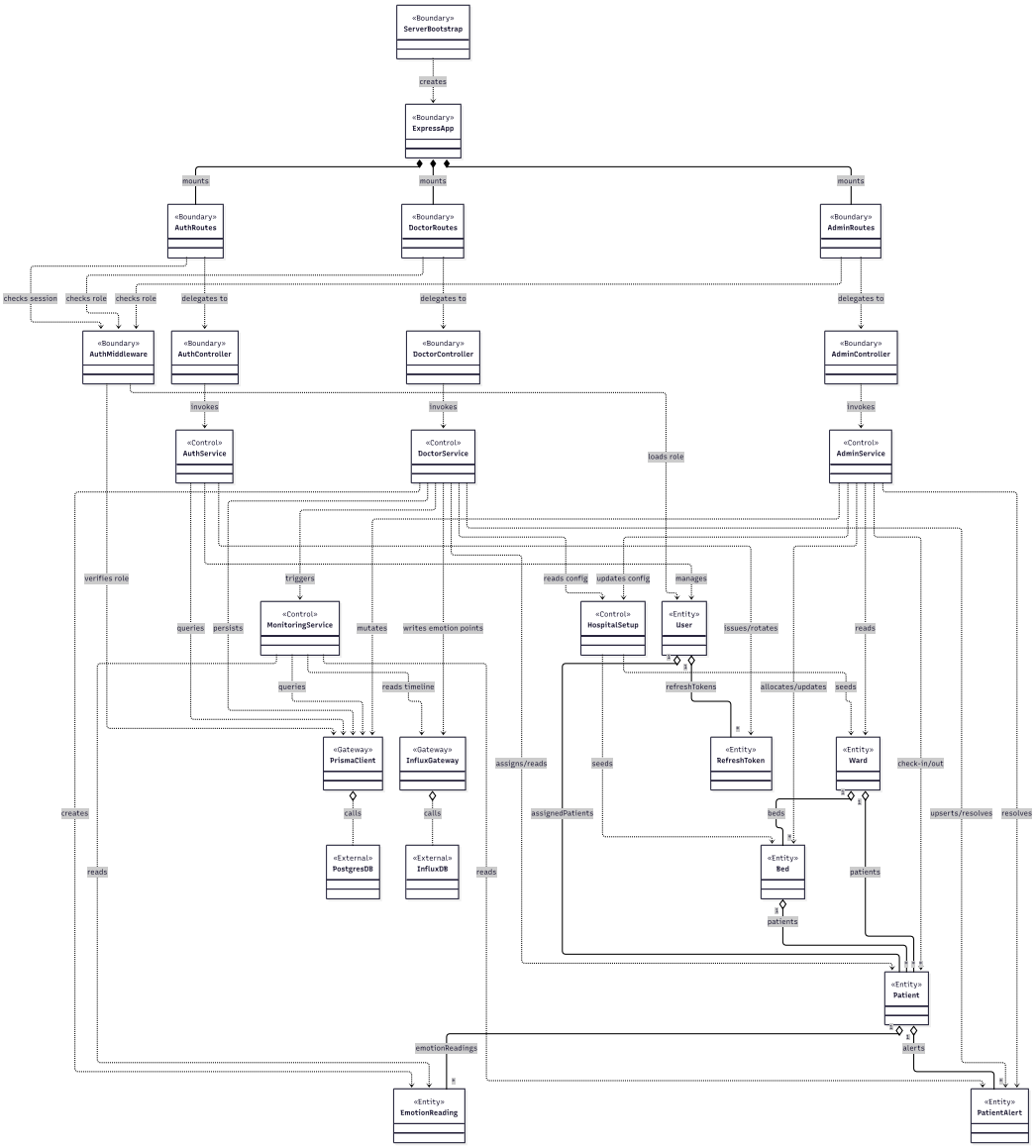
7.5 Modelul structural

7.5.1 Backend

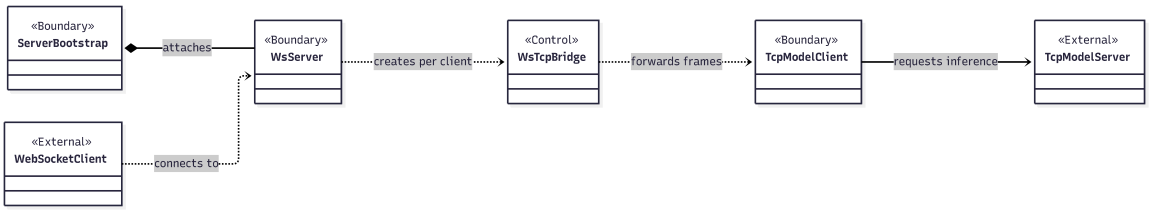
Figura 7.2a și Figura 7.2b ilustrează organizarea backend-ului bazată pe modelul arhitectural *Entity–Control–Boundary* (ECB), structură care asigură o separare clară a responsabilităților:

- *Entity* reprezintă nucleul domeniului, adică modelele persistente gestionate în baza de date (ex.: *User*, *Patient*, *EmotionReading*, *PatientAlert*).
- *Control* concentrează logica de business și orchestrează fluxurile de lucru (serviciile aplicației).
- *Boundary* delimitează interfața către exterior și gestionează comunicarea cu clienții (rute, controllere, middleware, servere de transport).

Pentru integrarea cu infrastructura, arhitectura este extinsă cu etichetele *Gateway* și *External*, care izolează accesul la servicii externe și decuplează logica internă de detaliile tehnice (de exemplu, accesul la baze de date). În Figura 7.2a, inițializarea aplicației este realizată de *ServerBootstrap* și *ExpressApp*, care montează rutele (*AuthRoutes*, *DoctorRoutes*, *AdminRoutes*) și aplică *AuthMiddleware* pentru validarea sesiunii și a rolurilor. La nivelul stratului de expunere HTTP, componentele de tip controller (AuthController, DoctorController, AdminController) au fost proiectate conform paradigmei de 'thin controllers' (asigurând o logică minimală de procesare) și deleagă operațiile către servicii (*AuthService*, *DoctorService*, *AdminService*), astfel încât regulile de business și validările specifice domeniului să fie centralizate și ușor de întreținut. Serviciile operează asupra entităților domeniului (de exemplu, *AuthService* gestionează *User*/*RefreshToken*, *DoctorService* gestionează *Patient*/*EmotionReading*/*PatientAlert*, iar *AdminService* gestionează *Ward*/*Bed*/*Patient*).



(a) API-ul HTTP: Boundary (Express/Routes/Controllers/Middleware), Control (Services) și persistența prin Gateway/External.



(b) Canalul de inferență în timp real: WebSocket ↔ TCP (model extern).

Figura 7.2: Modelul structural al backend-ului, conform ECB (Entity–Control–Boundary) extins cu Gateway și External.

Persistența și integrarea sunt decuplate prin gateway-uri: *PrismaClient* mediază accesul la baza de date relațională (PostgreSQL), iar *InfluxGateway* asigură scrierea/citirea seriilor temporale în InfluxDB, pentru vizualizări și analiză istorică. Separat de fluxul HTTP, Figura 7.2b prezintă canalul de procesare în timp real pentru inferență: *WsServer* acceptă conexiuni de la clientul WebSocket al aplicației web, iar *WsTcpBridge* orchestrează fluxul dintre WebSocket și modelul extern, trimițând frame-urile prin *TcpModelClient* către *TcpModelServer*. Rezultatele inferenței (probabilități) sunt retransmise către client, păstrând responsabilitățile de transport separate de logica de business și de mecanismele de stocare.

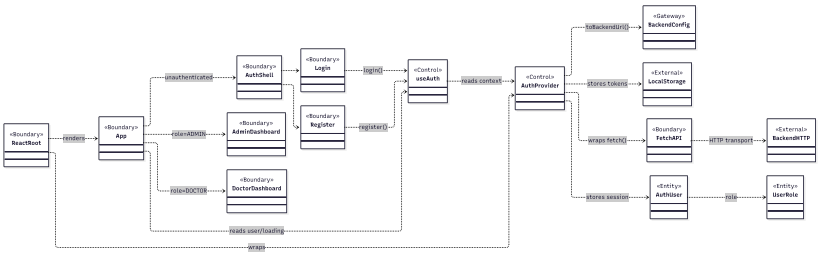
7.5.2 Frontend

Figura 7.3 prezintă structura frontend-ului (React + TypeScript) folosind aceeași separare ECB ca în backend, însă cu o nuanță importantă: *Entity* reprezintă în principal **modele de date/DTO-uri și stări UI** (de ex. *AuthUser*, *DoctorPatientSummary*, *MonitoringPoint*), nu entități persistate direct în aplicația client. *Control* grupează logica de orchestrare (hook-uri precum *useAuth*, *useEmotionData* și provider-ul *AuthProvider*), iar *Boundary* include paginile și componentele UI (de ex. *App*, *DoctorDashboard*, *CameraStreamer*, *LiveSpikeMonitor*). Pentru integrare, sunt evidențiate și componente *Gateway/External*, care izolează accesul la resurse de infrastructură (configurația către backend, transport HTTP/WS, API-uri ale browserului).

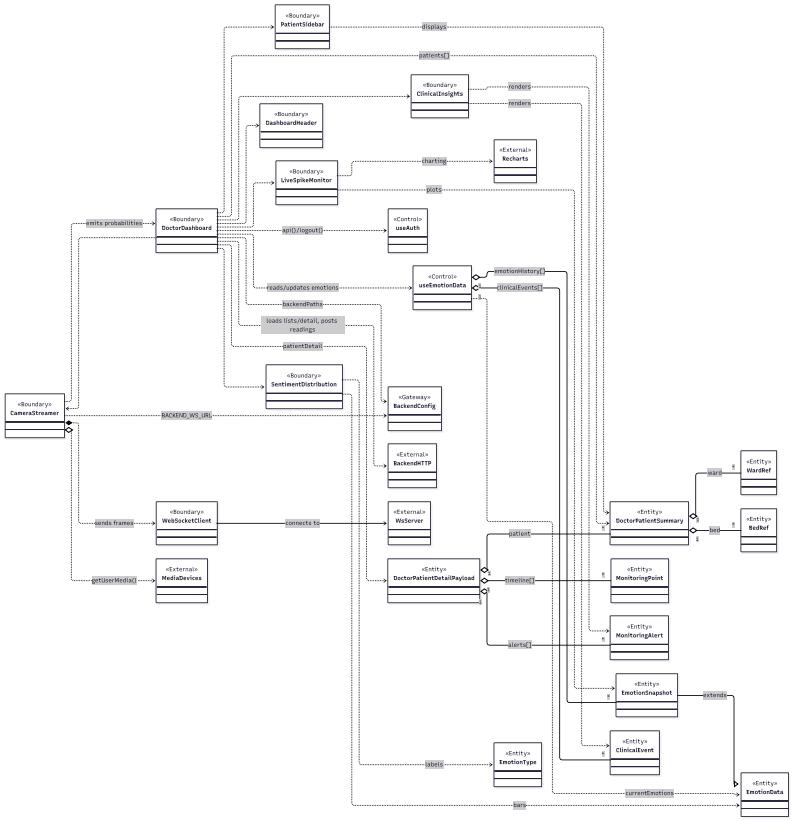
În Figura 7.3a este surprinsă fundația aplicației: *ReactRoot* compune *AuthProvider* peste *App*, iar aplicația decide ruta principală în funcție de sesiune și rol. *AuthProvider* centralizează autentificarea și persistă token-urile în *LocalStorage*, oferind un API unificat pentru cereri HTTP (prin *FetchAPI* și configurarea adreselor prin *BackendConfig*). Această separare reduce duplicarea și menține componentele de UI independente de detaliile de transport.

Figura 7.3b descrie fluxul principal al medicului: *DoctorDashboard* încarcă lista de pacienți și detaliile de monitorizare din backend (DTO-uri precum *DoctorPatientDetailPayload*, *MonitoringPoint*, *MonitoringAlert*), apoi compune componente specializate pentru vizualizare și suport decizional. Starea „live" a sesiunii este gestionată de *useEmotionData* (entități UI precum *EmotionData*, *EmotionSnapshot*, *ClinicalEvent*), în timp ce *CameraStreamer* folosește *MediaDevices* pentru captură și un *WebSocketClient* dedicat pentru trimiterea frame-urilor către backend. Probabilitățile primite sunt propagate către dashboard pentru actualizarea UI și, periodic, pentru persistență prin endpoint-urile HTTP (ex. trimiterea de readings).

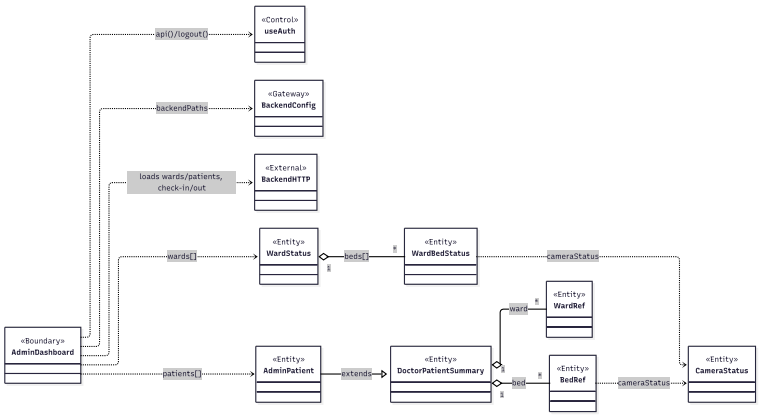
În Figura 7.3c este reprezentat fluxul administrativ, unde *AdminDashboard* gestionează operațiile de check-in/check-out și statusul secțiilor/paturilor folosind DTO-uri precum *WardStatus*, *WardBedStatus* și *AdminPatient*. Ca și în cazul medicului,



(a) Bootstrap și autentificare (App Shell).



(b) Fluxul medicului: monitorizare, vizualizare și streaming live.



(c) Fluxul administrativ: management secții/paturi/pacienți.

Figura 7.3: Modelul structural al frontend-ului, conform ECB (Entity–Control–Boundary), extins cu Gateway și External.

comunicarea cu backend-ul este abstractizată prin *useAuth* și aceeași infrastructură de transport/configurare, menținând UI-ul concentrat pe logică de prezentare și pe interacțiunile specifice rolului.

7.5.3 Diagrama entitate–relație (ER)

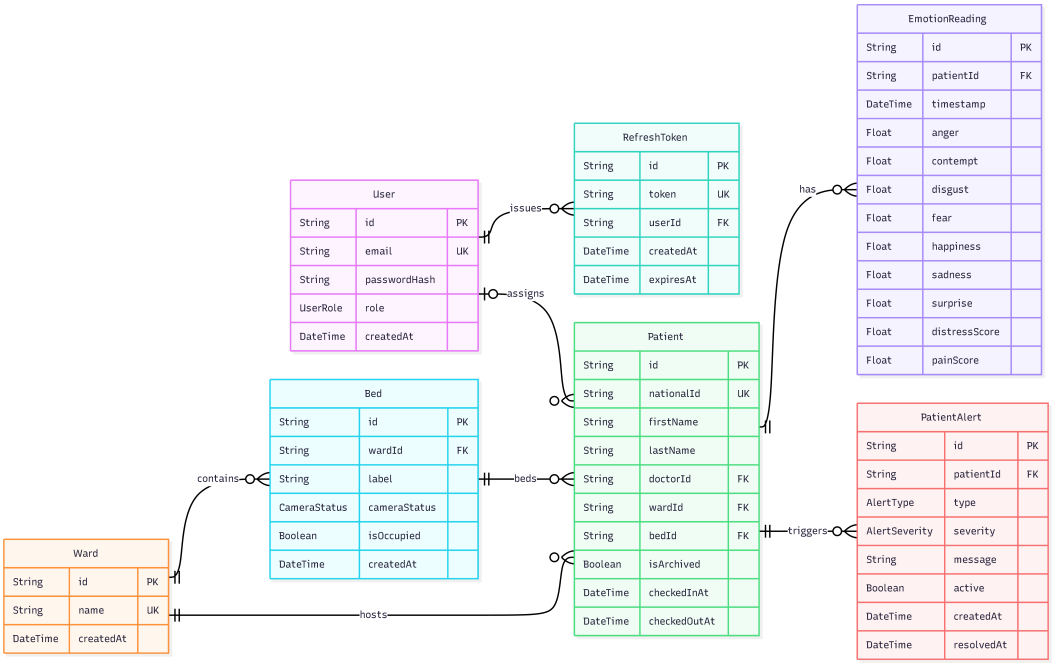


Figura 7.4: Diagrama entitate–relație (ER) a bazei de date.

Figura 7.4 prezintă modelul relațional al datelor persistente utilizate de aplicație, complementând diagramele ECB. Dacă în modelul structural *Entity* apare ca „nucleu al domeniului", diagrama ER detaliază modul în care acest nucleu este implementat în baza de date: chei primare, chei străine, constrângeri de unicitate și relațiile dintre entități (cardinalități).

Zona de autentificare este compusă din *User* și *RefreshToken*: fiecare utilizator poate emite mai multe refresh token-uri (relația *User–RefreshToken*), iar câmpul *role* (*UserRole*) diferențiază fluxurile și permisiunile din aplicație (medic vs. administrator). La nivel operațional, aceste entități susțin sesiuni persistente și controlul accesului pe roluri.

Modelarea spitalului și a monitorizării este structurată în jurul entității *Patient*, care este asociată cu o secție (*Ward*) și un pat (*Bed*), iar opțional poate fi asignată unui medic (*User*). Starea clinică în timp este capturată prin *EmotionReading* (seturi de valori pe emoții + scoruri agregate), legate de pacient, iar evenimentele de risc sunt reprezentate prin *PatientAlert* (tip, severitate, activ/rezolvat). În acest fel, baza de date relațională păstrează atât structura organizațională (secții/paturi/pacienți),

cât și istoricul și semnalările necesare fluxurilor din interfața medicului și a administratorului.

7.6 Modelul dinamic al sistemului

7.6.1 Diagrama de secvență: Monitorizare live

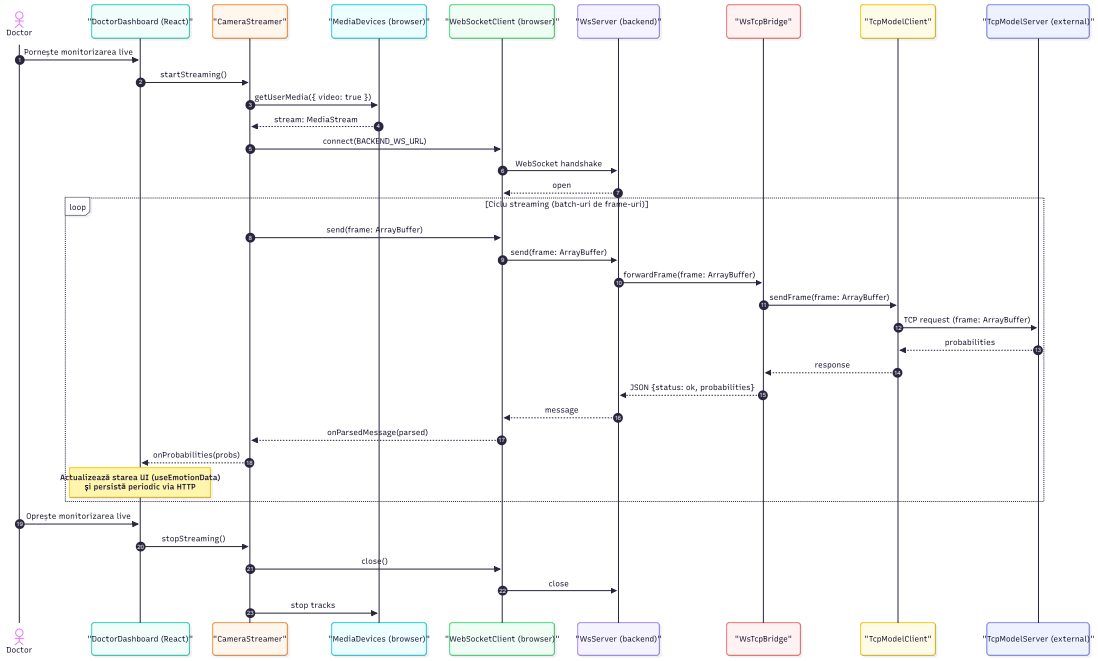


Figura 7.5: Diagrama de secvență pentru use case-ul *Monitorizare live*: captură video, transmitere prin WebSocket și inferență în model extern (TCP).

Figura 7.5 descrie fluxul de monitorizare în timp real, declanșat de medic din interfața aplicației. La pornirea sesiunii, componenta de captură inițializează camera folosind API-ul browserului (*MediaDevices*) și stabilește o conexiune WebSocket către backend. Cadrele video sunt pre-procesate (redimensionare/compresie) și transmise ca mesaje binare către server, permițând un canal cu latență redusă față de un flux exclusiv HTTP.

Pe backend, serverul WebSocket primește cadrele și le direcționează către un modul de tip „bridge” care gestionează ordonarea și trimiterea lor către modelul extern. Bridge-ul asigură conectarea către serviciul de inferență (prin clientul TCP) și aplică un mecanism de coadă, astfel încât procesarea să rămână stabilă chiar și când rata de captură depășește temporar capacitatea modelului. Rezultatul inferenței (probabilități pe emoții) este apoi serializat și retransmis prin WebSocket către aplicația web.

La nivel de UI, probabilitățile primite sunt folosite pentru actualizarea stării de

monitorizare (grafice, distribuții, evenimente clinice) fără a bloca interacțiunea utilizatorului. Oprirea monitorizării închide conexiunea WebSocket și eliberează resursele de captură (oprirea track-urilor camerei), închizând complet bucla de streaming.

7.6.2 Diagrama de comunicare: Vizualizare timeline

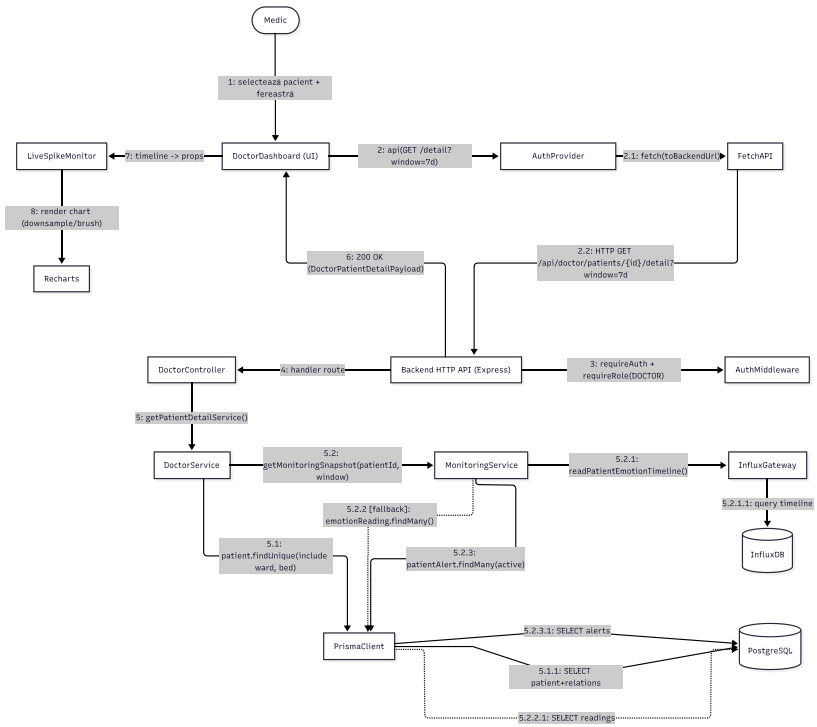


Figura 7.6: Diagrama de comunicare pentru use case-ul *Vizualizare timeline*.

Figura 7.6 evidențiază participanții și dependențele dintre ei pentru afișarea timelineului clinic. Interfața medicului declanșează cererea `/detail`, autorizată prin token JWT, proces prin care backend-ul orchestrează încărcarea metadatelor pacientului și a istoricului de monitorizare. Datele istorice, denumite `timelineHistory`, sunt preluate preferențial din *InfluxDB* via *gateway*, aplicându-se un mecanism de *fallback* pe *PostgreSQL* în cazul indisponibilității bazei de date temporale. La fiecare nouă inferență de emoții aceste date istorice sunt actualizate prin adăugarea ultimelor probabilități primite. Pentru a asigura continuitatea vizuală a graficului și o experiență de utilizare fluidă, sistemul implementează un mecanism de eșantionare locală la nivelul interfeței. Astfel, la fiecare 500 ms, buffer-ul local de memorie, `emotionHistory` — care reține datele din ultimele 65 de minute — este actualizat prin adăugarea de noi puncte de date. În intervalele în care nu sunt recepționate cadre noi de la motorul de inferență AI, valorile emoțiilor sunt menținute constante, corespunzând ultimelor probabilități identificate. Matematic, acest comportament

este echivalent cu un filtru de menținere de ordin zero (*Zero-Order Hold*), unde valoarea funcției de stare $E(t)$ rămâne invariantă între două momente de detecție succesive t_n și t_{n+1}:

$$E(t) = E(t_n), \quad \text{pentru} \quad t_n \leq t < t_{n+1} \tag{7.1}$$

În final, payload-ul vizualizat de medic reprezintă o concatenare (*mergedMonitorHistory*) a celor două fluxuri: `timelineHistory` (datele persistate) și `emotionHistory` (datele curente din memoria volatilă). Acest set de date unificat este mapat către componenta *LiveSpikeMonitor*, care utilizează biblioteca *Recharts* pentru randarea graficului în timp real, oferind o rezoluție de 0.5 secunde. Un exemplu vizual intuitiv este oferit în Figura 7.7. Alertele actualizate sunt folosite de mecanismul de notificări pentru a anunța doctorul schimbări importante legate de starea emoțională a pacientului.



Figura 7.7: Exemplu de vizualizare timeline.

7.6.3 Diagrama de deployment

Figura 7.8 evidențiază distribuția fizică a sistemului: aplicația web rulează în browser (interfața React) și comunică cu backend-ul Node.js atât sincron prin REST/HTTP (pentru operații CRUD și încărcarea datelor), cât și asincron prin WebSocket (pentru streaming de cadre și rezultate). Backend-ul accesează persistența relațională în PostgreSQL prin Prisma și utilizează InfluxDB ca depozit time-series pentru timeline-uri, iar inferența este externalizată către un server de model dedicat, accesat prin TCP. Între TCP și WS există o punte de legătură (*TsTcpBridge*) ce asigură comunicarea între cele 2 protocoale.

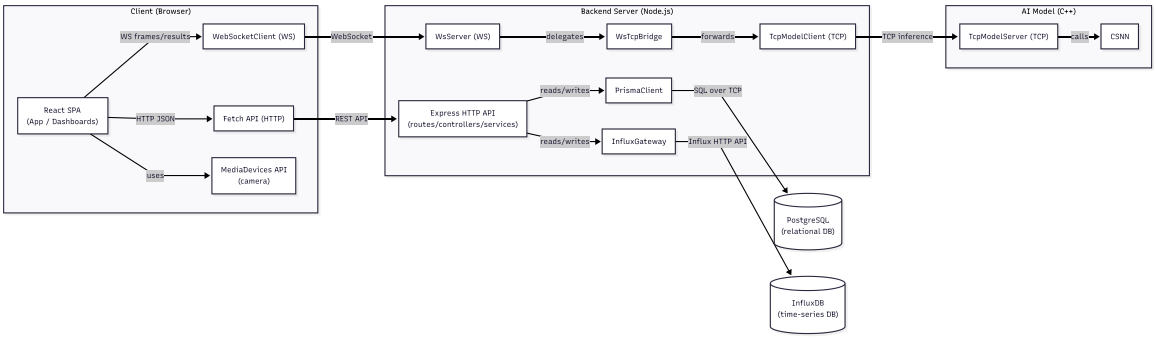


Figura 7.8: Diagrama de deployment: noduri de execuție și protocoale de comunicare (REST/HTTP, WebSocket, TCP).

Motivația utilizării PostgreSQL + InfluxDB

Pentru persistență am ales o abordare hibridă, separând datele **tranzacționale** (structurate, cu relații) de datele **time-series** (flux continuu de puncte indexate temporal). Această separare reduce cuplarea dintre workload-ul de tip OLTP (operare zilnică, integritate, actualizări) și workload-ul de tip analiză temporală (range queries, agregări pe ferestre).

- **PostgreSQL (bază relațională, sursa de adevăr pentru domeniu clinic).** PostgreSQL este utilizat pentru entitățile cu relații clare și reguli de consistență: utilizatori și roluri, pacienți, alocarea în salon/pat, stări operaționale, respectiv alertele persistente. Acest tip de date beneficiază direct de: *ACID transactions*, constrângeri (chei primare/străine), modelare normalizată, interogări cu *JOIN*-uri și migrații controlate ale schemei. Într-un context clinic, aceste proprietăți sunt importante pentru a evita stări invalide (ex. pacient fără pat/ward valid sau alerte asociate greșit) și pentru a păstra un istoric coerent.

- **InfluxDB (TSDB pentru timeline-uri și interogări pe intervale de timp).** Pentru monitorizarea emoțiilor, aplicația produce un volum mare de puncte cu timestamp (emoții + scoruri derivate), interogate aproape exclusiv pe ferestre de timp (ex. ultimele 24h sau 7 zile) și afișate ca grafice. O bază de date time-series este optimizată pentru acest tip de acces: (i) scrieri frecvente și eficiente, (ii) compresie și stocare eficientă pentru date temporale, (iii) interogări rapide de tip *range scan* ("toate punctele dintre t_0 și t_1"), (iv) posibilitatea de a aplica politici de retenție și/sau downsampling fără a afecta modelul relațional.

Eficiență și izolare a performanței. Deși datele time-series pot fi păstrate și într-o bază relațională, pe măsură ce crește frecvența de eșantionare și retenția, tabelele mari pot introduce costuri suplimentare (indexuri mai voluminoase, operații de mentenanță, impact asupra latențelor OLTP). Prin utilizarea InfluxDB pentru

timeline-uri, încărcarea asociată interogărilor temporale este mutată către un motor dedicat, menținând backend-ul și baza relațională responsive pentru operațiile critice (CRUD și tranzacții).

Robustețe operațională (fallback). Arhitectura permite ca timeline-ul să fie citit cu prioritate din InfluxDB (dacă este disponibil și are puncte), iar în caz contrar din persistența relațională; astfel, sistemul rămâne funcțional chiar și în situații de configurare incompletă sau indisponibilitate temporară a depozitului time-series.

7.6.4 Diagrama de stare: *PatientAlert*

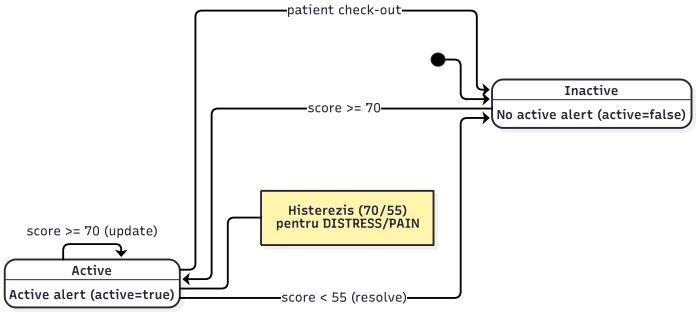


Figura 7.9: Diagrama de stare pentru *PatientAlert* (per pacient și tip de alertă).

Figura 7.9 descrie ciclul de viață al unei alerte clinice (*DISTRESS/PAIN*) asociate unui pacient. Pentru optimizarea mecanismului de declanșare a alertelor și eliminarea fenomenului de instabilitate (*jitter*) la pragurile de decizie, a fost implementat un algoritm de **histerezis** detaliat în Figura 7.10. Acesta utilizează praguri diferențiate pentru activarea ($T_{high} = 70$) și dezactivarea ($T_{low} = 55$) stării de alertă, asigurând astfel o stabilitate temporală a monitorizării și reducând oscilațiile frecvente ale stării.

Calculul severității se bazează pe două metrici fundamentale, derivate din vectorul stărilor emoționale detectate (V). Prima metrică, scorul de **Distress** (S_D), este calculată ca medie aritmetică a stărilor de furie, frică și tristețe, conform ecuației (7.2):

$$S_D = \text{clamp}\left(\frac{V_{\text{anger}} + V_{\text{fear}} + V_{\text{sadness}}}{3}, 0, 100\right) \tag{7.2}$$

Cea de-a doua metrică, scorul de **Pain** (S_P), reprezintă o medie ponderată ce prioritizează frica și tristețea ca indicatori non-verbali ai suferinței fizice, fiind definită în relația (7.3):

$$S_P = \text{clamp}\left(0.45 \cdot V_{\text{fear}} + 0.4 \cdot V_{\text{sadness}} + 0.15 \cdot V_{\text{disgust}}, 0, 100\right) \tag{7.3}$$

În scenariul în care ambele scoruri calculate prin formulele (7.2) și (7.3) depășesc pragul critic de activare (T_{high}), sistemul aplică o logică de arbitrare prin selecția

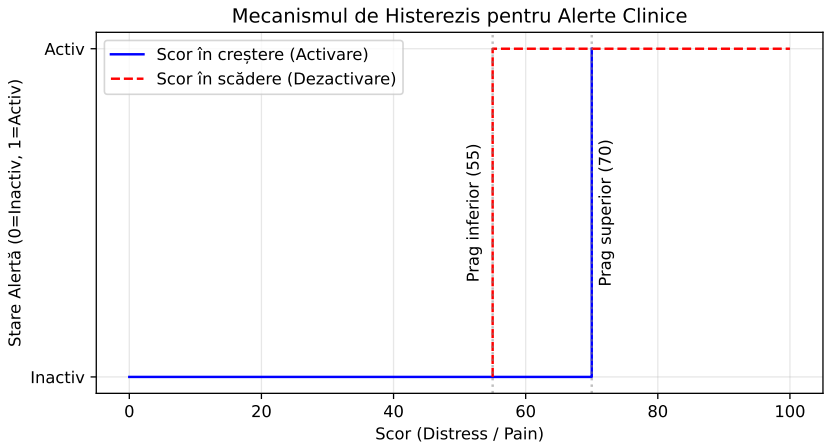


Figura 7.10: Reprezentarea grafică a mecanismului de histerezis aplicat sistemului de alerte. Activarea se produce la depășirea pragului de 70, în timp ce revenirea la starea inactivă necesită scăderea scorului sub 55.

valorii maxime. Astfel, intensitatea și eticheta alertei sunt determinate de scorul final (S_{final}), determinat conform relației (7.4):

$$S_{final} = \max(S_D, S_P) \tag{7.4}$$

O alertă trece în starea *Active* atunci când valoarea rezultată din ecuația (7.4) respectă condiția $S_{final} \geq 70$. În timpul stării active, severitatea și mesajul sunt actualizate dinamic pe baza noilor citiri. Pentru a evita declanșarea repetitivă a notificărilor la fluctuații minore, rezolvarea alertei se produce doar atunci când $S_{final} < 55$, moment în care alertarea este dezactivată și se înregistrează timpul de rezolvare. Toate aceste actualizări au loc la persistarea unui nou reading de emoții, backendul calculând severitatea pe baza scorului:

- `HIGH` dacă score ≥ 85,
- `MEDIUM` dacă score ≥ 72,
- `LOW` în caz contrar.

În interfață, aceste alerte sunt afișate în *Clinical Insights* la secțiunea *Current Alerts* (Figura 7.11) și reprezintă starea clinică persistată pentru pacient (pe intervalele recente de funcționare).

Sistemul calculează și alerte **derivate** (nepersistate), pe baza unui *timeline* de emoții pe o fereastră de timp (ex. 24h sau 7d). Aceste alerte urmăresc episoade susținute, nu doar vârfuri scurte:

- Se generează o alertă de tip `DISTRESS` sau `PAIN` dacă scorul corespunzător rămâne peste **65%** timp de cel puțin **60 minute**.

- Mesajul include durata episodului și vârful (peak) atins în acea perioadă.
- Severitatea este calculată după aceeași logică (HIGH/MEDIUM/LOW) în funcție de peak.

În interfață, aceste alerte apar în *Clinical Insights* la secțiunea *Prolonged Alerts* (Figura 7.12) și sunt recalculte la fiecare obținere a unui nou *snapshot* de monitorizare din backend.

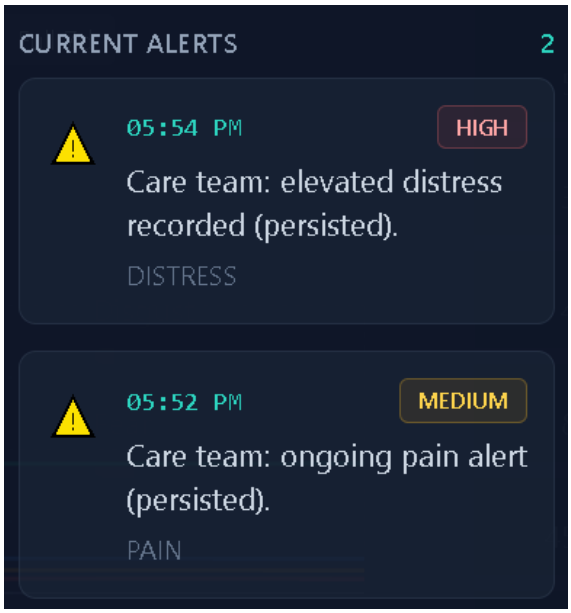


Figura 7.11: Secțiunea *Current Alerts* - vizualizarea alertelor persistente.

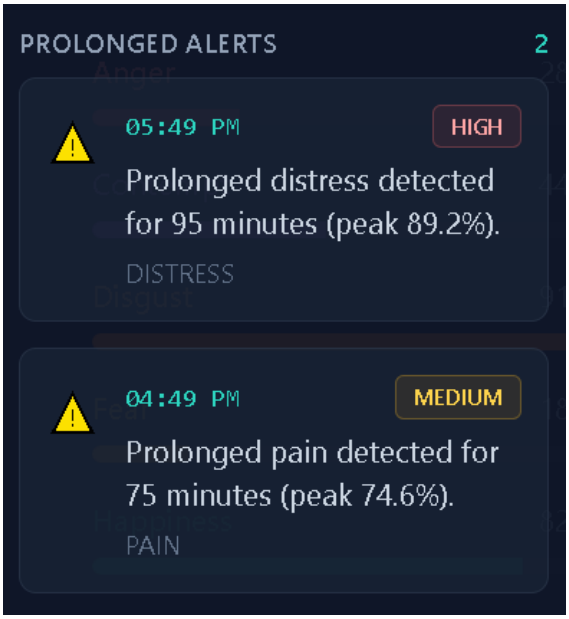


Figura 7.12: Secțiunea *Prolonged Alerts* - vizualizarea alertelor de durată.

7.6.5 Diagrama de activități: Check-in pacient

Figura 7.13 prezintă fluxul operațional de internare (check-in) și deciziile principale care garantează consistența datelor: validarea câmpurilor obligatorii, verificarea existenței patului, compatibilitatea pat–secție (când se indică o secție), și prevenirea ocupării simultane. Operația este tratată tranzacțional, astfel încât înregistrarea/actualizarea pacientului și marcarea patului ca ocupat să fie aplicate atomic. În cazul unui pacient deja internat se returnează un răspuns de tip conflict, iar pentru erori neprevăzute se semnalizează eșecul operației.

7.7 Sistemul de alerte și notificări

Aplicația integrează un mecanism multi-nivel de alertare, destinat în principal utilizatorului cu rol de medic. Semnalele sunt generate atât **în timp real** (din probabi-

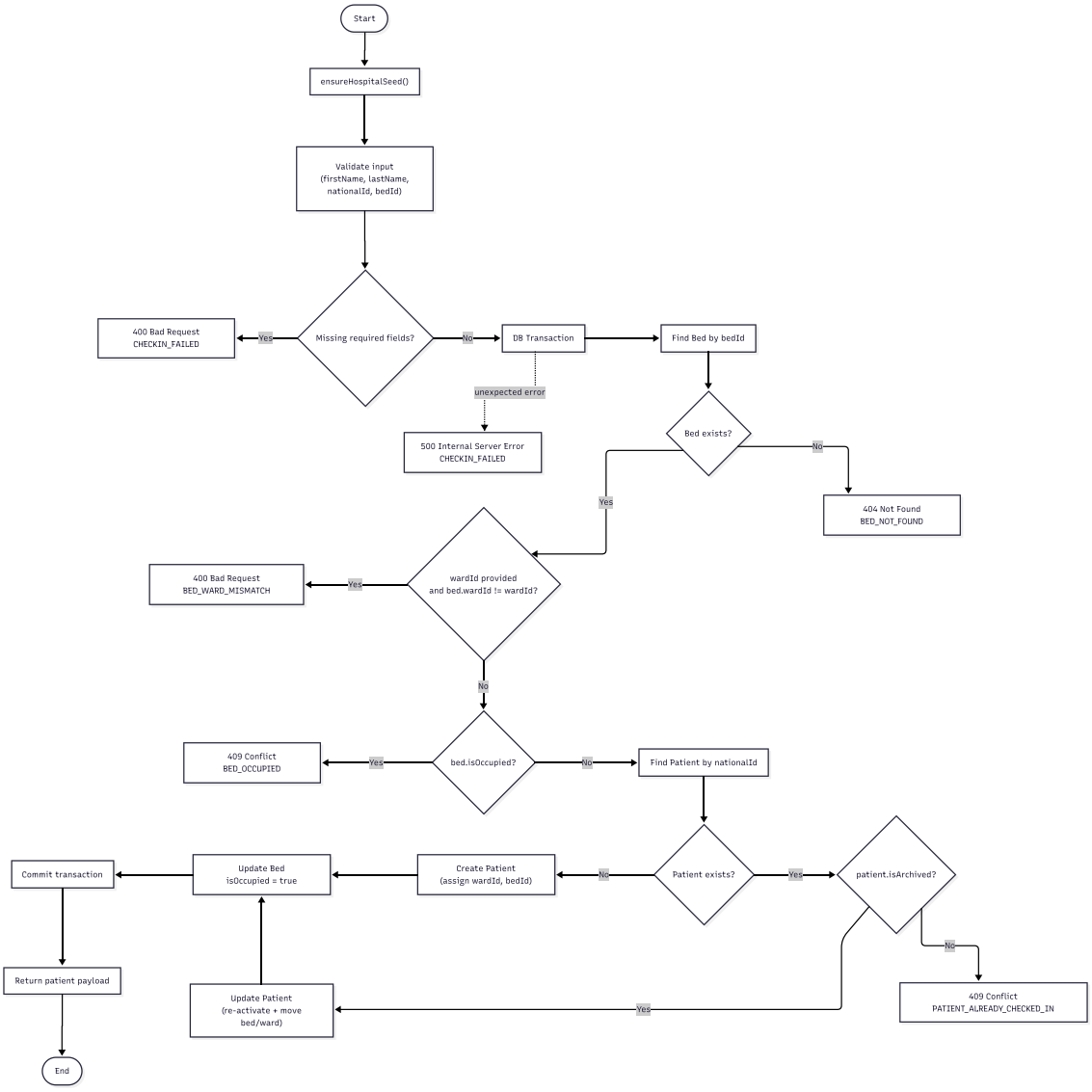


Figura 7.13: Diagrama de activități pentru use case-ul *Check-in pacient*.

litățile inferate de model), cât și **la nivel backend** (din istoricul pacientului și alertele persistate). În plus, există notificări operaționale (erori/stări de funcționare).

7.7.1 Evenimente clinice în sesiune (Live Session Events)

La recepționarea fiecărui vector de probabilități de la modelul de inferență, valorile emoționale sunt supuse unui proces de normalizare în intervalul $[0, 100]$, fiind ulterior redate în interfața medicului sub formă de procente. Această vizualizare a emoțiilor curente, exemplificată în Figura 7.14, oferă o analiză granulară a profilului afectiv, permițând identificarea rapidă a stărilor dominante și a nuanțelor emoționale ale pacientului. Simultan cu afișarea brută a datelor, sistemul execută un mecanism de analiză euristică pentru identificarea episoadelor emoționale semnificative. Se determină emoția dominantă (E_{max}) prin identificarea valorii maxime din setul de probabilități curente. Dacă această valoare depășește pragul critic de activare, stabilit la un nivel de încredere de 60%, sistemul generează automat un eveniment clinic (*Clinical Event*), conform logicii descrise în relația (7.5):

$$\text{Trigger} = \begin{cases} \text{generare eveniment}, & \text{daca } \max(V_{\text{emotii}}) > 60 \\ \text{stare latenta}, & \text{altfel} \end{cases} \tag{7.5}$$

Structura și comportamentul acestor evenimente sunt definite prin următoarele caracteristici tehnice, așa cum reiese și din detaliile surprinse în figurile de mai jos:

- **Trigger**: $\max(\text{emotii}) > 60\%$.
- **Conținut**: emoția dominantă, intensitatea (în %), timestamp și un mesaj selectat aleator din 3 variante predefinite (pentru a evita repetitivitatea).
- **Afișare**: evenimentele apar în panoul *Clinical Insights* la secțiunea *Live Session Events*.
- **Feedback vizual**: emoția detectată este evidențiată temporar (efect de tip *flash/pulse*).

7.7.2 Indicatori rezumați în UI

Clinical Risk Level (Low / Medium / High)

În *Patient Sidebar* se afișează nivelul rezumat de risc clinic (*Clinical Risk Level*), derivat din media emoțiilor negative:

$$\text{avgNegative} = \frac{\text{Anger} + \text{Fear} + \text{Sadness}}{3} \tag{7.6}$$



Figura 7.14: Vizualizarea emoțiilor și a intensităților curente.

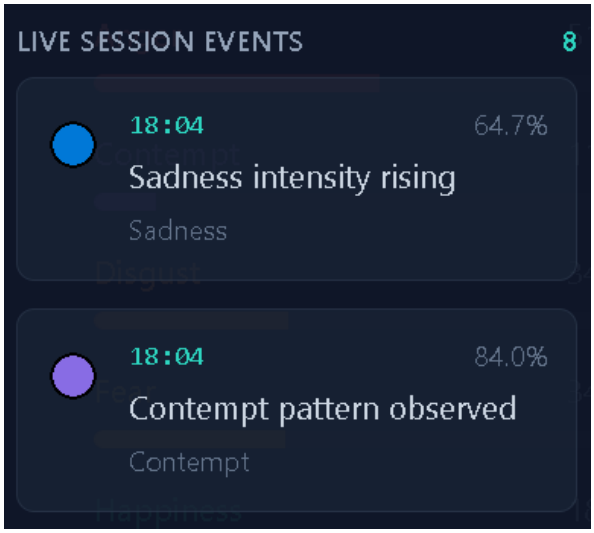


Figura 7.15: Detaliu al panoului *Clinical Insights* cu evenimentele generate.

$$\text{riskLevel} = \begin{cases} \text{High}, & \text{dacă avgNegative} > 50 \\ \text{Medium}, & \text{dacă avgNegative} > 25 \\ \text{Low}, & \text{altfel} \end{cases} \tag{7.7}$$

Exemplele vizuale pentru cele trei niveluri sunt prezentate în Figura 7.16.



(a) Low (b) Medium (c) High

Figura 7.16: Nivelul de risc clinic (Clinical Risk Level) în cele trei stări.

Session Type (normal / positive / high-risk)

În header-ul aplicației (lângă titlul platformei) este afișat *Session Type*, care sintetizează rapid starea sesiunii. Acesta este determinat combinând alertele clinice și emoțiile curente:

- **High-risk**: dacă există cel puțin o alertă cu severitate `HIGH` (din *Current* sau *Prolonged Alerts*) **sau** dacă Anger > 60 și Fear > 60.
- **Positive**: dacă Happiness > 50 și Surprise > 30.

- **Normal**: în celelalte situații.

Exemplele vizuale pentru cele trei tipuri sunt prezentate în Figura 7.17.



(a) Normal (b) Positive (c) High-risk

Figura 7.17: Tipul sesiunii (*Session Type*) afișat în header, în cele trei stări.

7.7.3 Notificare la schimbarea nivelului de risc

Sistemul urmărește dinamica lui *riskLevel* și notifică medicul **doar când există o schimbare**:

- **Risk Increased** (degradare): când nivelul trece de la Low → Medium/High sau Medium → High.
- **Risk Decreased** (îmbunătățire): când nivelul scade (High → Medium/Low sau Medium → Low).
- Dacă nivelul rămâne neschimbat, **nu se afișează notificare**.

Exemplele vizuale ce apar timp de 5 secunde în partea dreaptă de sus pentru cele două situații sunt prezentate în Figura 7.18.



(a) Notificare de degradare (*Risk Increased*).

(b) Notificare de îmbunătățire (*Risk Decreased*).

Figura 7.18: Notificarea afișată la schimbarea nivelului de risc.

7.7.4 Notificări operaționale (erori și stări de funcționare)

Pe lângă alertarea clinică, aplicația comunică și stări/erori operaționale prin mesaje vizuale:

- **Autentificare**: erori la login/înregistrare (mesaj afișat în formular).
- **Operații de administrare/doctor**: erori la încărcarea listelor sau la acțiuni (ex. preluare pacient, check-in/check-out), afișate ca mesaj de eroare în UI.

- **Stare conexiune către model**: indicatori de tip *connected/disconnected*, respectiv stări de captură (*recording/idle*), pentru a confirma funcționarea pipeline-ului cameră–model.

7.8 Testarea aplicației

7.8.1 Testare white-box

Metoda analizata

Analiza white-box a fost realizată pe metoda `derivePrologedAlert` din serviciul de monitorizare, care se ocupă cu detectarea perioadelor de suferință prelungită (durere sau stres) ale unui pacient, analizând un istoric de date (timeline). Această funcție include ramificatii multiple, mai multe iterații și o funcție auxiliară (`flushStreak`) care influențează direct decizia de generare a alertelor.

Complexitate ciclomatrică

Pentru metoda analizată, au fost identificate 8 predicate de decizie (D), care determină ramificarea fluxului de execuție, după cum se poate remarca în Tabelul 7.2.

ID	Condiție (Predicat)	Descriere / Rol în fluxul de execuție
D_1	`timeline.length === 0`	Verificarea listei vide pentru *early return*.
D_2	`timeline.forEach`	Controlul iterațiilor prin punctele cronologiei.
D_3	`value >= threshold`	Condiția de depășire a pragului (început/continuare *streak*).
D_4	`streakStart < 0`	Identificarea inițierii unui segment nou de alertă.
D_5	`streakStart < 0`	Verificarea existenței unui *streak* activ la apelul `flushStreak`.
D_6	`streakStart < 0`	Verificarea *streak*-ului rămas deschis după finalizarea buclei.
D_7	`durationMinutes >= 60`	Validarea duratei minime necesare pentru generarea alertei.
D_8	`durationMinutes >= 60`	Validarea duratei pentru *streak*-ul procesat la finalul execuției.

Tabela 7.2: Predicatele de decizie identificate pentru funcția de alertare.

Determinarea complexității ciclomatrice (CC) se bazează pe teoria grafurilor aplicată fluxului de control al programului. Relația fundamentală utilizată este formula lui McCabe, introdusă de Thomas McCabe în lucrarea sa fundamentală din 1976 [McC76], fiind definită în ecuația 7.8:

$$CC = E - N + 2P \tag{7.8}$$

Unde variabilele reprezintă:

- E – numărul de muchii (edges) din graful de flux;
- N – numărul de noduri (nodes);
- P – numărul de componente conexe (pentru o singură metodă, $P = 1$).

Aplicând această metodologie asupra grafului de flux de control (CFG) generat - ilustrat în Figura 7.19, complexitatea a fost calculată prin trei metode convergente:

1. $CC =$ numărul de regiuni planare ale grafului $= 9$
2. $CC = D + 1 = 8 + 1 = 9$
3. Conform ecuației 7.8: $CC = 37 - 30 + 2 * 1 = 9$

Drumuri independente (set minim teoretic, CC=9)

Pentru CFG-ul metodei, complexitatea ciclomatrica este $CC = 9$, deci exista 9 drumuri independente teoretice care trebuie parcurse în faza de testare unitară pentru a asigura o acoperire completă la nivel de decizie. În Tabelul 7.3 este un set practic de drumuri independente descrise prin semnătura deciziilor $D_1 \dots D_8$.

Maparea testelor pe drumuri independente (P1–P9)

Pentru funcția analizată, setul de teste white-box, ilustrat în Tabelul 7.4, a fost construit astfel încât fiecare drum independent din CFG să fie acoperit de cel puțin un test.

$$\text{Path coverage} = \frac{9}{9} = 100\%$$

Rezultatul confirmă acoperirea completă a setului minim de drumuri independente identificat prin complexitatea ciclomatrică.

7.9 Testarea de integrare

7.9.1 Testare de integrare (Top-down, Depth-first)

Validarea sistemului a fost realizată printr-o abordare incrementală de tip *Top-down, Depth-first*. Această metodă a permis verificarea timpurie a fluxului critic de date,

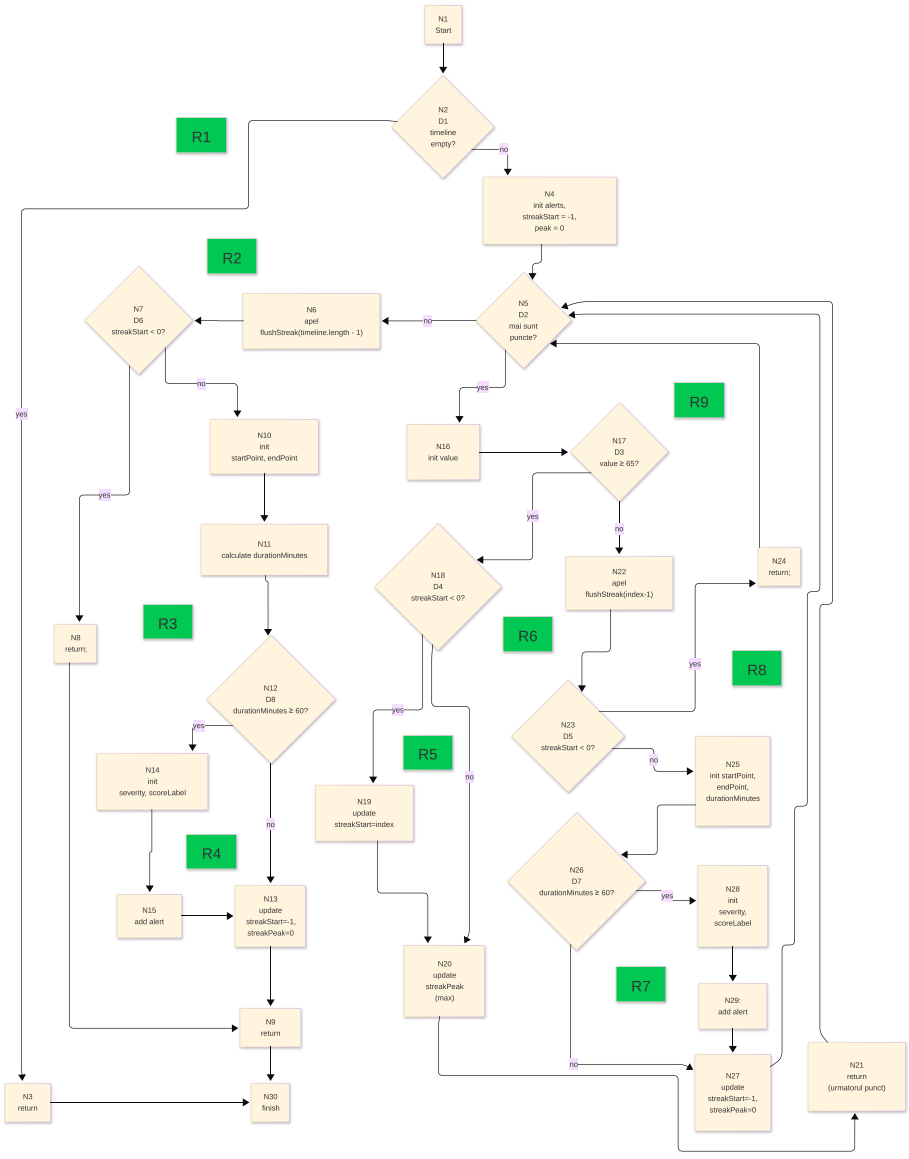


Figura 7.19: CFG pentru `derivateProlongedAlert`.

Tabela 7.3: Maparea corectată a cazurilor de testare (White-box)

Scenariu	**Descriere**	**Condiții Logice Corecte**
R1	Early return (Timeline gol)	$D_1 = T$
R2	Final cu streak lung (Alertă creată la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = T$
R3	Nicio activitate (loop terminat, niciun streak găsit)	$D_1 = F, D_2 = F, D_6 = T$
R4	Final cu streak scurt (Fără alertă la flush-ul final)	$D_1 = F, D_2 = F, D_6 = F, D_8 = F$
R5	Start Streak Nou (Prima valoare ≥ 65)	$D_1 = F, D_2 = T, D_3 = T, D_4 = T, D_2 = F, D_6 = T$
R6	Ignorare valoare mică (Sub prag, dar nu eram în streak)	$D_1 = F, D_2 = T, D_3 = F, D_5 = T, D_2 = F, D_6 = T$
R7	Continuare Streak (Valoare ≥ 65, dar deja în streak)	$D_1 = F, D_2 = T, D_3 = T, D_4 = F, D_2 = F, D_6 = T$
R8	Închidere streak lung (Sub prag, alertă creată în buclă)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = T, D_2 = F, D_6 = T$
R9	Închidere streak scurt (Sub prag, streak-ul nu are 60 min)	$D_1 = F, D_2 = T, D_3 = F, D_5 = F, D_7 = F, D_2 = F, D_6 = T$

Tabela 7.4: Corespondența testelor white-box pe drumuri (input / output așteptat)

Test	**Drum**	**Timeline (minute, scor)**	**Tip**	**Prag**	**Output**
P1	R1	[]	DISTR.	65	[]
P2	R6 → R3	$[(0, 20)]$	DISTR.	65	[]
P3	R5 → R4	$[(0, 70)]$	DISTR.	65	[]
P4	R5 → R7 → R2	$[(0, 70), (65, 80)]$	DISTR.	65	1 alertă; t_{65}
P5	R5 → R7 → R8 → R3	$[(0, 70), (75, 80), (80, 20)]$	DISTR.	65	1 alertă; t_{75}
P6	R5 → R7 → R9 → R3	$[(0, 70), (30, 80), (45, 20)]$	DISTR.	65	[]

Notă: În coloana „Drumuri asociate", am listat succesiunea de ramificări logice prin care trece execuția. De exemplu, R5 → R7 → R2 înseamnă: deschidere streak în buclă , actualizare peak la a doua iterație și generare alertă la ieșirea finală din funcție.

asigurându-se că integrarea componentelor respectă arhitectura definită și că datele circulă corect de la interfața HTTP până la nivelul de persistență.

Obiectiv și Strategie

Obiectivul principal a fost validarea completă a endpoint-ului `POST /api/doctor/patients/:patientId/readings`, care reprezintă nucleul funcțional al aplicației (colectarea de telemetrie și generarea de alerte). Strategia adoptată a presupus integrarea progresivă a componentelor conform următorului lanț de dependențe:

$$\text{App} \rightarrow \text{Route} \rightarrow \text{Middleware} \rightarrow \text{Controller} \rightarrow \text{Service} \rightarrow \{\text{Prisma/Postgres}, \text{InfluxDB}\}$$

Procesul a fost structurat în cinci etape distincte, detaliate în **Tabelul 7.5**. Fiecare pas a vizat extinderea razei de testare prin înlocuirea progresivă a componentelor simulate (*stubs* și *mocks*) cu implementări reale, până la obținerea unui sistem complet funcțional.

Tabela 7.5: Strategie Depth-first: pași, focus și componente simulate

Step	Nivel introdus	Ce testezi efectiv (Focus)	Componente simulate
1	Entry Point & Security	App + Route + Middleware. Verificarea autentificării și autorizării la nivel de rută.	Controller (Stub)
2	Orchestrare API	Integrarea Controller-ului: extragerea parametrilor și delegarea către logica de business.	Service (Stub)
3	Business Logic	Integrarea Service-ului real: calculul scorului de distress și logica de declanșare a alertelor.	Prisma + Influx (Mocks)
4	Data Access	Integrarea cu Postgres (via Prisma): persistența reală a citirilor și alertelor.	Influx (Mock)
5	Time-series / Telemetry	Integrarea cu InfluxDB: flux complet de scriere și citire a metricilor pe axa temporală.	Fără (Sistem complet)

Validarea Fluxului de Date (Input/Output)

Pentru fiecare etapă definită anterior, au fost stabilite seturi riguroase de date de intrare și rezultate așteptate. Această trasabilitate asigură faptul că nicio componentă nouă introdusă în lanț nu alterează logica validată anterior.

Corelarea dintre input-ul simulat și răspunsul sistemului pentru fiecare pas de integrare este sintetizată în **Tabelul 7.6**. Se observă tranziția de la un simplu răspuns de tip *hook* (Step 1) către o validare complexă a stării bazei de date și a consistenței telemetriei (Step 5).

Tabela 7.6: Input și output așteptat pe pașii de integrare

Step	Input (rezumat)	Output așteptat (rezumat)
1	Request HTTP `POST` cu JWT valid.	Status `202` și confirmarea atingerii rutei.
2	Request HTTP cu payload valid.	Status `201` și apelarea service-ului cu parametrii corecți.
3	Payload ce induce scor de distress ridicat.	Status `201` și validarea logică a calculului de alertă.
4	Request cu Postgres activ în container.	Status `201` și persistența confirmată în tabelele SQL.
5	Secvență `POST` urmată de `GET`.	Status `200` și timeline consistent extras din InfluxDB.

Mediu de Testare și Infrastructură

Pentru a garanta fidelitatea testelor față de mediul de producție, s-a renunțat la bazele de date *in-memory* în favoarea unor instanțe reale rulate în containere. Suita de teste utilizează următoarele instrumente:

- **Jest & Supertest:** Cadru de testare și bibliotecă pentru simularea cererilor HTTP fără ridicarea manuală a serverului.
- **Testcontainers:** Gestionarea automată a ciclului de viață pentru containerele Docker (`postgres:16-alpine` și `influxdb:2.7-alpine`).
- **Prisma Migrations:** Sincronizarea automată a schemei bazei de date înainte de execuția testelor prin `prisma migrate deploy`.

Această infrastructură permite izolarea completă a fiecărui test, asigurând un mediu curat (prin mecanisme de *cleanup* post-test) pentru fiecare iterație a strategiei *depth-first*.

7.10 Manual de utilizare al aplicației

7.10.1 Scopul aplicației și roluri

Aplicația **Neuro-Guardian** este o platformă web de monitorizare clinică asistată de analiză emoțională, destinată utilizării în context spitalicesc. Utilizarea este organizată pe două roluri:

- **ADMIN** (personal administrativ): gestionează internarea/externarea pacienților și alocarea lor pe secție/pat.
- **DOCTOR** (personal medical): preia pacienți disponibili și urmărește tabloul de monitorizare (vizualizări, alerte, trenduri).

7.10.2 Accesarea aplicației

Aplicația se accesează dintr-un browser modern (Chrome/Edge/Firefox), prin adresa pusă la dispoziție (de ex. `http://localhost:5173/` în rulare locală). După autentificare, interfața este afișată automat în funcție de rol (ADMIN sau DOCTOR).

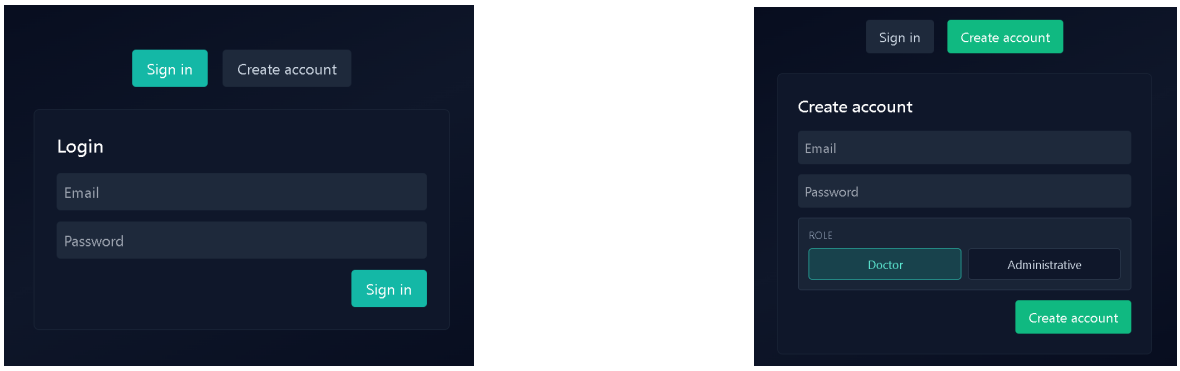
7.10.3 Autentificare și creare cont

Autentificare (Sign in)

Pentru autentificare se introduce adresa de e-mail și parola, apoi se apasă butonul *Sign in*. În cazul unor date invalide, aplicația afișează un mesaj de eroare.

Creare cont (Create account)

Pentru înregistrare se introduc e-mailul și parola, apoi se selectează rolul (*Doctor* sau *Administrative*) și se apasă *Create account*. Într-un scenariu real, conturile cu rol administrativ pot fi restricționate conform politicilor instituției.



(a) Interfața de autentificare (b) Interfața de creare cont

Figura 7.20: Ecranele de autentificare și creare cont.

7.10.4 Modul ADMIN: Administrative Dashboard

După autentificare cu rolul ADMIN, utilizatorul accesează *Administrative Dashboard*, care include următoarele funcționalități principale:

Reîmprospătare și deconectare

În partea superioară există:

- **Refresh**: reîncarcă listele de secții/paturi și pacienții activi.
- **Logout**: închide sesiunea curentă.

Internare pacient (Patient Check-in)

Secțiunea *Patient Check-in* permite înregistrarea unui pacient activ în sistem:

1. Se completează *First name*, *Last name* și *National ID / CNP*.
2. Opțional, se completează *Diagnosis* și *Medical history*.
3. Se selectează *Ward* (secția) și *Bed* (patul disponibil).
4. Se apasă *Check in patient*.

Aplicația nu permite alocarea unui pat deja ocupat (paturile disponibile sunt filtrate automat).

Pacienți activi (Active Patients) și externare (Check-out)

În *Active Patients* se afișează pacienții internați (activi), împreună cu secția/patul și momentul internării. Pentru externarea unui pacient se apasă butonul *Check-out* corespunzător.

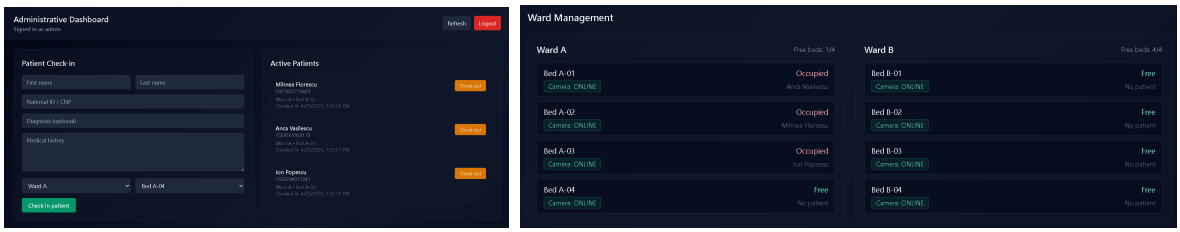
Management secții/paturi (Ward Management)

Secțiunea *Ward Management* oferă o vedere de ansamblu asupra:

- gradului de ocupare (paturi libere/ocupate),
- stării camerei asociate patului (*ONLINE/OFFLINE/MAINTENANCE*),
- pacientului asociat (dacă patul este ocupat).

7.10.5 Modul DOCTOR: Doctor Panel și Monitoring Dashboard

După autentificare cu rolul DOCTOR, utilizatorul poate gestiona pacienții disponibili și poate accesa tabloul de monitorizare.



(a) Dashboard administrativ (b) Managementul secției și paturilor

Figura 7.21: Modul ADMIN: internare/externare și managementul secției.

Lista pacienților și preluare (Take patient)

În *Doctor Panel* sunt afișate două liste:

- **Available Patients**: pacienți nealocați; pot fi preluați prin *Take patient*.
- **My Patients**: pacienți alocați doctorului; pot fi selectați pentru monitorizare.

Butonul *Refresh patients* reîncarcă listele. Dacă nu există pacienți disponibili/ai doctorului, aplicația afișează mesaje de tip *No unassigned patients right now* sau *Pick a patient from the available list to start monitoring*.

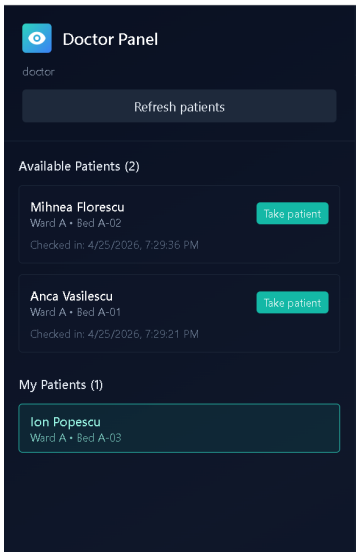


Figura 7.22: Modul DOCTOR: preluarea și selectarea pacienților.

Tabloul de monitorizare (Monitoring Dashboard)

După selectarea unui pacient, aplicația afișează tabloul de monitorizare, care include:

- **Header de sesiune**: status (*Normal Session*, *High Risk: Agitation*, *Positive Engagement*), butoanele *Live Monitor*, *Insights*, afișare/ascundere panou și *Logout*.

- **Patient Sidebar**: informații despre pacient și nivelul de risc curent (Low/Medium/High).
- **Zona principală**: flux camera (stream) și distribuția emoțiilor curente.

Live Monitor (vizualizare istoric și spike-uri)

Prin butonul *Live Monitor* se deschide o fereastră peste ecran (overlay) care prezintă un grafic cu istoricul emoțiilor, o figură reprezentativă fiind 7.7. Fereastra se închide cu butonul *Close* sau prin click în afara zonei.

Insights (alerte și interpretare clinică)

Prin butonul *Insights* se deschide un panou lateral cu:

- alerte curente și alerte prelungite (prolonged),
- evenimente clinice derivate (listate cronologic).

Scopul este de a sintetiza situațiile relevante (ex. creșteri susținute ale indicatorilor negativi).

Notificări de risc

La schimbarea nivelului de risc, aplicația afișează o notificare scurtă (*Risk Increased / Risk Decreased*) cu tranziția dintre niveluri. Notificarea dispare automat după câteva secunde.

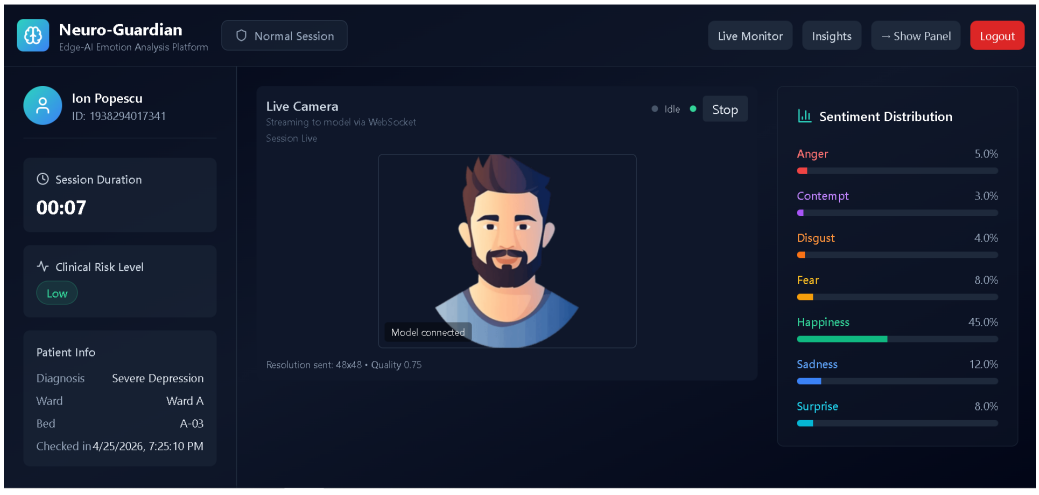


Figura 7.23: Modul DOCTOR: tabloul de monitorizare și instrumentele de analiză.

7.10.6 Confidențialitate și bune practici

Aplicația gestionează informații sensibile; se recomandă utilizarea în rețele securizate, păstrarea credențialelor confidențiale și folosirea *Logout* pe dispozitive partajate.

7.11 Proiectare

7.11.1 Analiza cerințelor

7.11.2 Diagrama cazurilor de utilizare

7.11.3 Diagrame de interacțiune

7.12 Implementare

7.12.1 Server

7.12.2 Client

7.13 Testare

Capitolul 8

Concluzii

Concluzii ...

Bibliografie

[BAA+23] Sami Barchid, Benjamin Allaert, Amel Aissaoui, José Mennesson, and Chaabane C Djeraba. Spiking-fer: Spiking neural network for facial expression recognition with event cameras. In *Proceedings of the 20th International Conference on Content-Based Multimedia Indexing (CBMI '23)*, pages 1–7, 2023.

[BBJ+19] Zhenshan Bing, Ivan Baumann, Zhuangyi Jiang, Kai Huang, Caixia Cai, and Alois Knoll. Supervised learning in snn via reward-modulated spike-timing-dependent plasticity for a target reaching vehicle. *Frontiers in Neurorobotics*, 13:10, 2019.

[BL73] Timothy VP Bliss and Terje Lømo. Long-lasting potentiation of synaptic transmission in the dentate area of the anaesthetized rabbit following stimulation of the perforant path. *The Journal of physiology*, 232(2):331–356, 1973.

[BP98] Guo-qiang Bi and Mu-ming Poo. Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, 18(24):10464–10472, 1998.

[Bur06] Anthony N Burkitt. A review of the integrate-and-fire neuron model: I. homogeneous synaptic input. *Biological Cybernetics*, 95(1):1–19, 2006.

[Che17] Yanqing Chen. Mechanisms of winner-take-all and group selection in neuronal spiking networks. *Frontiers in computational neuroscience*, 11:20, 2017.

[DB92] Serena M Dudek and Mark F Bear. Homosynaptic long-term depression in hippocampus induced by low-frequency stimulation. *Proceedings of the National Academy of Sciences*, 89(10):4363–4367, 1992.

[DC15a] Peter U. Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience*, 9:99, 2015.

[DC15b] Peter U Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in computational neuroscience*, 9:99, 2015.

[DPT01] Arnaud Delorme, Laurent Perrinet, and Simon J Thorpe. Networks of integrate-and-fire neurons using rank order coding b: Spike timing dependent plasticity and emergence of orientation selectivity. *Neurocomputing*, 38:539–545, 2001.

[EA24] Mireille El-Assal. *Unsupervised STDP-based Feature Learning for Video Analysis with Spiking Neural Networks*. Doctoral dissertation, Université de Lille, 2024.

[Fal19] Pierre Falez. *Improving Spiking Neural Networks Trained with Spike Timing Dependent Plasticity for Image Recognition*. PhD thesis, Université de Lille, 2019.

[FMT18] Paul Ferré, Franck Mamalet, and Simon J Thorpe. Unsupervised feature learning with winner-takes-all based stdp. *Frontiers in computational neuroscience*, 12:24, 2018.

[FTB+19] Pierre Falez, Pierre Tirilly, Ioan Marius Bilasco, Philippe Devienne, and Pierre Boulet. Multi-layered spiking neural network with target timestamp threshold adaptation and stdp. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2019.

[GFES21] Wenzhe Guo, Mohammed E. Fouda, Ahmed M. Eltawil, and Khaled Nabil Salama. Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems. *Frontiers in Neuroscience*, 15, 2021.

[GK02] Wulfram Gerstner and Werner M Kistler. *Spiking neuron models: Single neurons, populations, plasticity*. Cambridge university press, 2002.

[GM08] Tim Gollisch and Markus Meister. Rapid neural coding in the retina with relative spike latencies. *Science*, 319(5866):1108–1111, 2008.

[Gru23] Marcus Grum. Learning representations by crystallized backpropagating errors. In *Artificial Intelligence and Soft Computing - 22nd International Conference, ICAISC 2023, Zakopane, Poland, June 18-22, 2023, Proceedings, PartI*, Lecture Notes in Computer Science, pages 78–100, 2023.

[Heb02] Donald O Hebb. *The Organization of Behavior: A Neuropsychological Theory*. 2002.

[HH52] Alan L Hodgkin and Andrew F Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 1952.

[HLD21] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. v2e: From video frames to realistic dvs events. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1312–1321, 2021.

[HS81] Berthold KP Horn and Brian G Schunck. Determining optical flow. *Artificial intelligence*, 17(1-3):185–203, 1981.

[HZ23] Manu Srinath Halvagal and Friedemann Zenke. The combination of hebbian and predictive plasticity learns invariant object representations in deep sensory networks. *bioRxiv*, pages 2023–03, 2023.

[Izh04] Eugene M Izhikevich. Which model to use for cortical spiking neurons? *IEEE Transactions on Neural Networks*, 15(5):1063–1070, 2004.

[Kas14] Nikola K. Kasabov. Neucube: A spiking neural network architecture for mapping, learning and understanding of spatio-temporal brain data. *Neural Networks*, 52:62–76, 2014.

[KGTM18] Saeed Reza Kheradpisheh, Mohammad Ganjtabesh, Simon J. Thorpe, and Timothée Masquelier. STDP-based spiking deep convolutional neural networks for object recognition. *Neural Networks*, 99:56–67, 2018.

[KPPC20] Alexander Kugele, Thomas Pfeil, Michael Pfeiffer, and Elisabetta Chicca. Efficient processing of spatio-temporal data streams with spiking neural networks. *Frontiers in Neuroscience*, 14:439, 2020.

[LBY20] D. Liu, N. Bellotto, and S. Yue. Deep spiking neural network for video-based disguise face recognition based on dynamic facial movements. *IEEE Transactions on Neural Networks and Learning Systems*, 31(6):1843–1855, 2020.

[LDP16] Jun Haeng Lee, Tobi Delbruck, and Michael Pfeiffer. Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, 10:508, 2016.

[LK81] Bruce D Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. *IJCAI*, 1981.

[LPSR18] Chankyu Lee, Priyadarshini Panda, Gopalakrishnan Srinivasan, and Kaushik Roy. Training deep spiking convolutional neural networks with stdp-based unsupervised pre-training followed by supervised fine-tuning. *Frontiers in Neuroscience*, 12:435, 2018.

[LWG+19] Dahua Li, Zhe Wang, Qiang Gao, Yu Song, Xiao Yu, and Chuhan Wang. Facial expression recognition based on electroencephalogram and facial landmark localization. *Technology and Health Care*, 27:1–15, 01 2019.

[LXJL22] Mengting Lan, Xiaogang Xiong, Zixuan Jiang, and Yunjiang Lou. PC-SNN: Supervised learning with local hebbian synaptic plasticity based on predictive coding in spiking neural networks. *CoRR*, abs/2211.15386, 2022.

[MBY21] E. Mansouri-Benssassi and J. Ye. Generalisation and robustness investigation for facial and speech emotion recognition using bio-inspired spiking neural networks. *Soft Computing*, 25:1717–1730, 2021.

[McC76] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.

[QBG11] Damien Querlioz, Olivier Bichler, and Christian Gamrat. Simulation of a memristor-based spiking neural network immune to device variations. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1775–1781, 2011.

[SM07] Gilad Silberberg and Henry Markram. Disynaptic inhibition between neocortical pyramidal cells mediated by martinotti cells. *Neuron*, 53(5):735–746, 2007.

[SMA00] Sen Song, Kenneth D Miller, and Laurence F Abbott. Competitive hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9):919–926, Sep 2000.

[ST94] Jianbo Shi and Carlo Tomasi. Good features to track. In *1994 Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, 1994.

[STZP13] Christos health Sagonas, Georgios Tzimiropoulos, Stefanos Zafeiriou, and Maja Pantic. 300 faces in-the-wild challenge: The first facial landmark localization challenge. In *Proceedings of the IEEE international conference on computer vision workshops*, pages 397–403, 2013.

[TCKPS20] C. Tan, G. Ceballos, N. Kasabov, and N. Puthanmadam Subramaniyam. Fusionsense: Emotion classification using feature fusion of multimodal data and deep learning in a brain-inspired spiking neural network. *Sensors*, 20(18):5328, 2020.

[TG98] Simon Thorpe and Jacques Gautrais. Rank order coding. In *Computational Neuroscience*, pages 113–118. 1998.

[TGK+19] Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. Deep learning in spiking neural networks. *Neural Networks*, 111:47–63, 2019.

[VJ01] Paul Viola and Michael Jones. Robust real-time face detection. In *Proceedings eighth IEEE international conference on computer vision*, volume 2, pages 747–747, 2001.

[vRBT00] M. C. W. van Rossum, G. Q. Bi, and G. G. Turrigiano. Stable hebbian learning from spike timing-dependent plasticity. 20(23):8812–8821, 2000.

[WDZ+22] Binqiang Wang, Gang Dong, Yaqian Zhao, Rengang Li, Hongbin Yang, Wenfeng Yin, and Lingyan Liang. Spiking emotions: Dynamic vision emotion recognition using spiking neural networks. In *2022 International Conference on Algorithms, High Performance Computing and Artificial Intelligence (AHPCAI)*, pages 1–6, 2022.

[ZZL22] Dongcheng Zhao, Yi Zeng, and Yang Li. Backeisnn: A deep spiking neural network with adaptive self-feedback and balanced excitatory–inhibitory neurons. *Neural Networks*, 154:68–77, 2022.

[ZZPM16] Amir Zadeh, Rowan Zellers, Eli Pincus, and Louis-Philippe Morency. Mosi: Multimodal corpus of sentiment intensity and subjectivity analysis in online opinion videos. *ArXiv*, abs/1606.06259, 2016.